

APUNTESWEB · INGENIERÍA INFORMÁTICA



Tercer curso · 2.º cuatrimestre · 8 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

NOTA

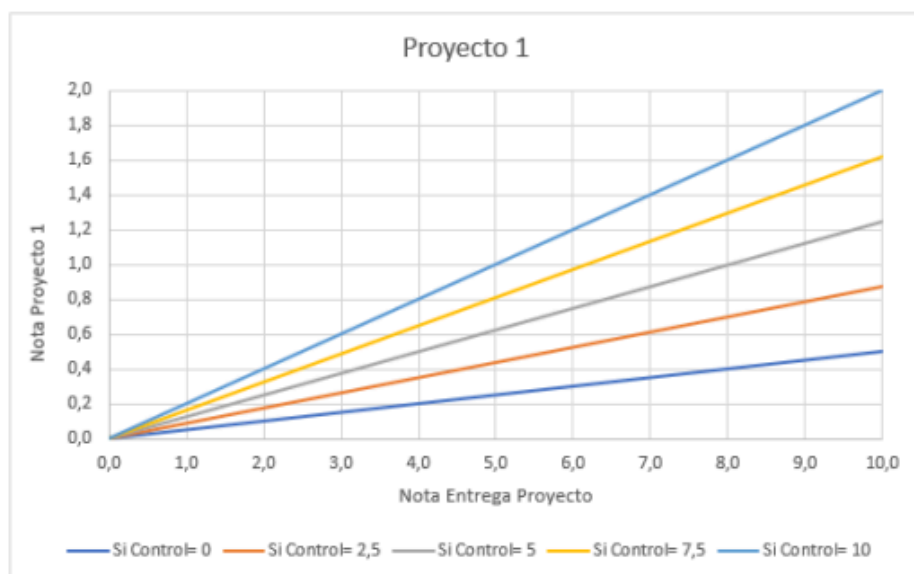
Quiero dejar esto de recuerdo para que quede constancia de lo que se sufría esta asignatura de puta mierda.

Proyecto 1 → HTML, CSS, JavaScript.

Evaluación → Entrega + Control

$$\text{Nota Proyecto 1} = A_0 \times \text{Entrega} \times [1 + A_1 \times (\text{Control} - 5)]$$

Nota.- En caso de obtener una nota para el Proyecto 1 < 0, el resultado final de 0 puntos.



1.1 Introducción a la evolución de la computación y la web

Evolución resumida

- **Años 40:** computación centralizada en grandes máquinas.
- **Finales de los 50:** varios terminales conectados a un sistema central.

- **Años 80:** expansión del ordenador personal.
- **Finales de los 80 y primera mitad de los 90:** auge de las **LAN**, que conectan equipos dentro de una red local para compartir recursos.
- **Segunda mitad de los 90:** expansión de **Internet** como red basada en estándares.
- **2007 en adelante:** la computación se desplaza al móvil.

La idea clave es que se pasa de sistemas aislados o locales a un entorno global donde se conectan ordenadores, servidores y otros dispositivos.

De cliente/servidor a la web

Las aplicaciones web siguen un modelo **cliente/servidor**:

- **Cliente:** lado del usuario.
- **Servidor:** lado donde se aloja y procesa la aplicación.

En la web, esto se traduce en:

- **navegador = frontend,**
- **servidor web = backend,**
- intercambio de recursos como **HTML** entre ambos.

Secuencia básica:

1. El usuario abre una web en el navegador.
2. El navegador solicita la página al servidor.
3. El servidor responde con HTML.
4. El navegador lo interpreta y lo muestra.

Qué abarcan las tecnologías web

Las **tecnologías web** son el conjunto de herramientas, lenguajes, protocolos y estándares que permiten crear y ejecutar servicios en la World Wide Web.

Frontend

- **HTML:** estructura.
- **CSS:** presentación visual.
- **JavaScript:** interactividad y lógica en el navegador.
- **Frameworks:** React, Angular, Vue.

Ejemplo rápido: HTML define un botón, CSS su aspecto y JavaScript lo que ocurre al pulsarlo.

Backend

- Se encarga de lógica, seguridad y acceso a datos.
- **Lenguajes / plataformas:** Java, PHP, Python, Node.js, .NET.

- **Frameworks:** Spring, Django, Laravel, Express.

Ejemplo: validar usuario y contraseña en un login.

Bases de datos

- **Relacionales:** MySQL, PostgreSQL.
- **NoSQL:** MongoDB.

Protocolos y estándares

- **HTTP/HTTPS:** comunicación cliente-servidor.
- **REST / GraphQL:** diseño de APIs.

Servidores web

- Apache
- Nginx
- Tomcat

1.2 Evolución de la tecnología web

Situación inicial.

Hasta los primeros años 2000, la interacción del usuario era reducida y se trabajaba principalmente con HTML, CSS y JavaScript básicos. La construcción de aplicaciones web se limitaba a:

- escribir y guardar código en un editor de texto plano,
- abrir localmente los ficheros en un navegador para comprobar su funcionamiento,
- desplegar los ficheros en un servidor web.

MPA (Multi-Page Application).

La **MPA** es el modelo tradicional de aplicación web en el que, cada vez que el usuario navega a una nueva ruta, el navegador carga una página HTML distinta desde el servidor.

Ejemplo: entrar en una tienda online y que cada sección cargue una página nueva completa.

Catalizador de 2004: más interacción y aparición de SPA.

En 2004, con el auge de las redes sociales y el nacimiento de Facebook, aumenta la interacción del usuario. Esto trae dos consecuencias principales:

- uso de tecnologías que no requieren recargar la página completa, como **AJAX**,
- necesidad de reducir ancho de banda y surgimiento de aplicaciones **SPA**.

SPA (Single Page Application).

Una **SPA** es una aplicación web que carga una sola página HTML y luego actualiza dinámicamente el contenido mediante JavaScript, sin volver a pedir páginas completas al servidor.

Ejemplo: Gmail o Trello, donde la interfaz cambia sin recargar toda la página.

Comparación entre MPA y SPA.

IMPORTANTE

Conceptos asociados:

- **SEO (Search Engine Optimization):** conjunto de técnicas para que una página aparezca más arriba en los buscadores de forma orgánica, sin pagar publicidad.
- **SSR (Server-Side Rendering):** técnica en la que las páginas se generan en el servidor y se envían al navegador ya renderizadas en HTML completo.

1.3 El impacto del móvil en la web

Catalizador de 2007: el móvil cambia la web.

En 2007 Apple presenta su iPhone 3 y la computación doméstica pasa al bolsillo de las personas. Las implicaciones que se remarcan son:

1. Nuevos formatos de pantalla incompatibles entre sí, lo que provoca la aparición de soluciones **responsivas**.
2. Nuevas versiones de navegadores que van incorporando etiquetas según se estandarizan.
3. Nuevas aplicaciones con contenido multimedia, redes sociales y geolocalización.
4. Aparición de las **PWA** para competir con apps móviles, gracias a la posibilidad de trabajar sin conexión cacheando recursos.

PWA (Progressive Web Application).

Una **PWA** es una aplicación web que usa tecnologías que permiten ofrecer una experiencia similar a una app nativa, pero sin necesidad de instalarla desde una tienda, porque funciona desde el navegador.

Ejemplos indicados en las diapositivas:

- Spotify web
- Uber móvil web
- Starbucks app web

Comparación: web normal vs PWA vs app nativa.

IMPORTANTE

Nota importante sobre PWA.

Al instalar una PWA en el PC, se instalan algunos archivos relacionados con el **Service Worker**, la **cache** y otros elementos, pero **nunca ficheros HTML o JS**. Esos archivos se almacenan en carpetas del usuario relativas al navegador.

1.4 Mobile-First y consecuencias

Catalizador de 2018: Mobile-First Indexing.

En 2018 Google cambia su algoritmo de posicionamiento hacia una política **Mobile-First Indexing**, debido a que el tráfico móvil superaba al de escritorio.

Consecuencia principal: la responsividad deja de ser recomendable y pasa a ser obligatoria.

Mobile-First Indexing (MFI) significa usar la versión móvil de una web como referencia principal para indexar y posicionar, en lugar de la versión de escritorio. Además, el mismo código HTML vale para móvil y para escritorio.

IMPORTANTE

Consecuencia general de la evolución web.

La consecuencia de toda esta evolución es que la codificación de una página web se vuelve **extremadamente compleja**. La solución que se destaca es el uso masivo de **JavaScript**, que pasa a ser protagonista del desarrollo de aplicaciones, junto con nuevos frameworks y librerías.

1.5 Frameworks, librerías y tecnologías de la asignatura

Frameworks y librerías destacados:

Tecnologías de la asignatura DAW.

Frontend:

- HTML
- CSS
- JavaScript
- jQuery
- Bootstrap

Backend:

- Java
- JSP

Servidor web:

- Apache
- Tomcat

2. HTML

2.1 Web, frontend, backend y publicación

La **Web** es un conjunto de documentos residentes en diferentes máquinas y relacionados mediante **hiperenlaces**.

Concepto	Idea clave
Frontend	Todo aquello con lo que el usuario interactúa. Incluye la página y lo que interpreta el navegador.
Página web	Documento que contiene el contenido a mostrar.
HTML	Lenguaje que estructura contenidos, enlaza páginas, muestra multimedia y permite visualizar la página en un navegador.
Navegador	Programa cliente que interpreta HTML.
Motor de renderizado	Parte del navegador que toma HTML, XML, imágenes y CSS para crear la representación visual.
Backend	Parte del sistema que atiende peticiones y sirve recursos desde el servidor.
Servidor web	Programa que recibe peticiones del cliente y sirve contenido estático: HTML, CSS, JavaScript e imágenes.
Motor HTTP	Recibe solicitudes HTTP/HTTPS, las interpreta, determina rutas, gestiona cabeceras, métodos y estados, y envía la respuesta.

HTML significa **HyperText Markup Language**. Fue propuesto en 1989 y combina:

- **universalidad**: un estándar común para todos;
- **lenguaje de marcas**: formato definido mediante etiquetas, concepto asociado a Tunncliffe en 1967;
- **hipertexto**: documentos enlazados mediante links, concepto asociado a Ted Nelson en 1965.

Navegadores y servidores

- Antes de 1993 los navegadores tenían poca funcionalidad.
- En 1993 Marc Andreessen desarrolla **Mosaic** en la NCSA.
- En 1994 aparece **Netscape**, con extensiones HTML propias como texto coloreado e imágenes.
- Microsoft responde con **Internet Explorer**, dando lugar a la guerra de navegadores.
- El **W3C**, impulsado por Tim Berners-Lee, busca ordenar el ecosistema y establecer estándares.

Publicación de una página

Para publicar una web se necesita subir el código a un **servidor propio** o contratar un **hosting**.

Una URL se interpreta como:

```
protocolo://nombredelsitioweb/rutahastalaraiz/ficherohtml
```

Ejemplo: `http://www.lawebdepepe.com/index.html`.

Instituciones y registros asociados:

- **ICANN**: Internet Corporation for Assigned Names and Numbers. Coordina a nivel global el sistema de nombres de dominio e IP para que Internet funcione de forma única y ordenada.
- **WHOIS**: directorio con información técnica y de contacto de propietarios de dominios. Permite consultar quién registró un dominio y ver datos asociados, como registrador, fechas, servidores DNS y contactos, cuando están disponibles.
- **DNRD**: Domain Name Registration Data. Es el conjunto de datos de registro de un dominio, como titular, registrador, estado, fechas y datos técnicos. WHOIS es una forma de consultar esos datos.

2.2 Historia, estandarización y versiones

Hitos principales

Fecha	Hito
1989	Tim Berners-Lee y Robert Cailliau presentan en el CERN la propuesta WorldWideWeb.
1991	Se publica <code>HTML Tags</code> , primer documento formal de HTML, con 22 etiquetas.
1993	El IETF intenta convertir HTML en estándar, pero la propuesta no fructifica.
1996	El W3C pasa a publicar los estándares HTML.
2004	Apple, Mozilla y Opera crean WHATWG para relanzar la estandarización.
Enero 2008	Primer <code>Working Draft</code> de HTML 5.0.
Diciembre 2012	Primera <code>Candidate Recommendation</code> de HTML 5.0.
Julio 2014	El W3C discute los Web Components , que permiten etiquetas personalizadas.

Evolución de versiones

Versión	Rasgos principales
HTML 1.0 / 2.0	Texto plano con hiperenlaces; sin adornos.
HTML 3.0	Etiquetas propietarias, colores, imágenes y guerra de navegadores.
HTML 3.2	Estandariza algunas extensiones propietarias y elimina otras.

Versión	Rasgos principales
HTML 4.0	Separa contenido, estructura y formato mediante CSS.
HTML 4.01	Exige que todo elemento de línea esté dentro de un elemento de bloque.
XHTML 1.0	Busca estructura XML; toda etiqueta abierta debe cerrarse.
HTML 5.0	Reduce compatibilidad con versiones obsoletas, facilita nuevos navegadores e incorpora nuevas etiquetas, atributos, APIs y recursos nativos.

Tipos de versiones:

- **estricta:** solo admite etiquetas aprobadas;
- **transitoria:** permite etiquetas no aprobadas;
- **conjunto de marcos:** permite etiquetas no aprobadas y marcos, actualmente desaconsejados por el W3C.

2.3 Documento HTML: estructura, sintaxis y metadatos

Estructura mínima

La estructura base de un documento HTML es:

- `<!DOCTYPE html>`: declara el tipo de documento;
- `html`: raíz del documento; debe incluir `lang` para declarar el idioma;
- `head`: metadatos, título, CSS, scripts y recursos no visibles como contenido principal;
- `title`: título de la pestaña, requerido y relevante para SEO;
- `body`: parte visible de la página.

En HTML5 la estructura se simplifica así:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Mi página</title>
</head>
<body>
  Contenido visible
</body>
</html>
```

DOCTYPE y validación

`<!DOCTYPE ...>` indica al navegador qué tipo de documento HTML está interpretando. En HTML5 se usa `<!DOCTYPE html>`.

Familia	Tipo	Declaración
HTML 4.01	Estricta	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd"></code>
HTML 4.01	Transitoria	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"></code>
HTML 4.01	Marcos	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN" "http://www.w3.org/TR/html4/frameset.dtd"></code>
XHTML 1.0	Estricta	<code><!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"></code>
XHTML 1.0	Transitoria	<code><!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"></code>
XHTML 1.0	Marcos	<code><!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd"></code>

El W3C permite validar páginas en `http://validator.w3.org`.

Codificación

HTML pretende ser texto universal, por eso debe declararse el conjunto de caracteres en `head`.

- Forma clásica: `<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />`.
- Forma HTML5: `<meta charset="utf-8">`.

Elementos, atributos y comentarios

Un elemento HTML tiene la forma:

```
<etiqueta atributo="valor">contenido</etiqueta>
```

Reglas prácticas:

- Todos los elementos pueden tener atributos, normalmente para añadir información o modificar su comportamiento.
- HTML no distingue mayúsculas/minúsculas en etiquetas ni atributos, pero se recomienda usar minúsculas.
- En XHTML las minúsculas y el cierre de etiquetas son obligatorios.
- Los valores de atributos pueden ir sin comillas, pero se recomienda usar `"..."`; también pueden usarse comillas simples si el valor contiene comillas dobles.
- Los atributos siempre van en la etiqueta de apertura y suelen ser pares `nombre="valor"`, aunque algunos son booleanos.

- Los **elementos vacíos** no tienen contenido ni etiqueta de cierre.
- `<tag/>` es estilo XHTML; `<tag>` como elemento vacío es la forma flexible habitual en HTML5.
- Los comentarios se escriben con `<!-- comentario -->`; pueden ser multilínea o aparecer en medio de una línea, y no se muestran en la página.

Atributo	Uso
<code>id</code>	Identifica un elemento único. Se usa en anclajes, CSS, JavaScript y asociación con <code>label</code> .
<code>class</code>	Agrupar elementos. Puede repetirse y un elemento puede tener varias clases separadas por espacios.
<code>name</code>	Nombra datos, sobre todo en formularios. No tiene que ser único; en <code>radio</code> agrupa opciones excluyentes.
<code>title</code>	Añade información emergente al pasar el ratón sobre el elemento.
<code>lang</code>	Declara el idioma del documento, útil para navegadores, lectores y buscadores.

Tipos de elemento:

- **documento:** estructuran la página (`html`, `head`, `body`);
- **bloque:** generan salto de línea y un rectángulo personalizable (tienen ancho y alto) (`h1`, `p`, `div`);
- **línea:** no generan salto de línea y ocupan solo lo necesario para mostrar su contenido (`a`, `img`, `span`);
- **vacíos:** sin contenido ni cierre (`img`, `br`, `meta`, según sintaxis HTML).

2.4 SEO y robots

Las diapositivas indican que hay más de 2 billones de páginas web y que se crean miles cada día. Para favorecer las visitas:

- usar palabras clave en `title`;
- usar palabras clave en encabezados como `h1`;
- usar texto alternativo `alt` en imágenes;
- añadir `<meta name="keywords" content="..." />`;
- añadir `<meta name="description" content="..." />`;
- enviar el sitio web a un motor de búsqueda.

Directivas para robots

Se indican con una metaetiqueta como:

```
<meta name="robots" content="noindex, nofollow" />
```

Directiva	Significado
<code>all</code>	Sin restricciones; es el valor predeterminado.

Directiva	Significado
<code>noindex</code>	No muestra la página ni enlace en caché en resultados.
<code>nofollow</code>	No sigue los enlaces de la página.
<code>none</code>	Equivale a <code>noindex, nofollow</code> .
<code>noarchive</code>	No muestra enlace en caché.
<code>nosnippet</code>	No muestra fragmento de la página.
<code>noodp</code>	No usa metadatos de Open Directory para títulos o fragmentos.
<code>notranslate</code>	No ofrece traducción en resultados.
<code>noimageindex</code>	No indexa imágenes de la página.
<code>unavailable_after</code>	No muestra la página después de la fecha indicada en formato RFC 850.

Robots citados: **Googlebot** y **MSNBot**.

2.5 Etiquetas de estructura, `head` y formato

`head`

Etiqueta	Función principal
<code>head</code>	Contenedor de metadatos; va entre <code>html</code> y <code>body</code> .
<code>title</code>	Título del documento en la pestaña; requerido.
<code>style</code>	Estilos definidos dentro de la página.
<code>link</code>	Relaciona el documento con recursos externos; también se usa para favicon, normalmente tras <code>title</code> .
<code>meta</code>	Define metadatos: <code>charset</code> ; <code>content</code> ; <code>http-equiv</code> como cabecera HTTP; <code>name</code> para <code>keywords</code> , <code>description</code> , <code>author</code> , <code>viewport</code> y robots.
<code>script</code>	JavaScript del lado cliente.
<code>base</code>	URL base o <code>target</code> de las URLs relativas; debe tener <code>href</code> y/o <code>target</code> .

`viewport` representa el área visible de la web y cambia según el dispositivo.

Semántica y estructura

`div` y `span` son contenedores genéricos: `div` de bloque y `span` de línea. En HTML5 se prefieren etiquetas semánticas cuando describen mejor el contenido.

Etiqueta	Uso
header	Cabecera de documento o sección: encabezados, logo/icono, autoría.
nav	Conjunto de enlaces de navegación.
main	Contenido principal del documento, normalmente único.
section	Grupo temático: capítulos, introducción, contacto, etc.
article	Contenido independiente y reutilizable; si contiene imágenes puede usar figure.
aside	Contenido relacionado indirectamente, como barra lateral.
footer	Pie con autoría, copyright, sitemap, vuelta arriba o documentos relacionados.
figure	Agrupar contenido visual o ilustrativo.
figcaption	Pie descriptivo de figure.

Formato y texto

Etiqueta	Uso
h1 ... h6	Encabezados jerárquicos; los buscadores los usan para indexar estructura y contenido.
p	Párrafo.
hr	Separador horizontal.
br	Salto de línea dentro de un bloque; no sustituye a p.
b	Negrita visual.
strong	Texto importante, normalmente en negrita.
i	Itálica visual.
em	Énfasis, normalmente en itálica.
mark	Texto resaltado.
small	Texto pequeño.
del	Texto borrado, normalmente tachado.
ins	Texto insertado, normalmente subrayado.
sub	Subíndice.
sup	Superíndice.
pre	Texto preformateado; conserva espacios y saltos.
blockquote	Cita extensa separada del texto.
q	Cita breve en línea.
abbr	Abreviatura.
address	Información de contacto, normalmente en itálica y bloque aparte.

Etiqueta	Uso
<code>cite</code>	Título de una obra, normalmente en itálica.
<code>bdo</code>	Cambia la dirección del texto con <code>dir="rtl"</code> o <code>dir="ltr"</code> .
<code>button</code>	Botón clicable; puede estar <code>disabled</code> y usar <code>type="button"</code> , <code>submit</code> o <code>reset</code> .

2.6 Enlaces, listas, imágenes y tablas

Enlaces

La etiqueta `a` crea hiperenlaces. Su destino se indica con `href`, que puede apuntar a una página, archivo, correo o ancla interna.

Caso	Forma
Enlace externo	URL de otro sitio.
Enlace interno	Página del mismo sitio.
Ancla interna	<code>href="#id"</code> apunta al elemento con ese <code>id</code> .
Imagen como enlace	Se coloca <code>img</code> dentro de <code>a</code> .
Botón como enlace	Puede usarse <code>onclick=document.location="link"</code> .

El atributo `target` controla dónde se abre el enlace:

- `_self`: misma ventana o pestaña, valor por defecto;
- `_blank`: nueva ventana o pestaña;
- `_parent`: frame padre;
- `_top`: ventana completa.

Listas

Etiqueta	Uso
<code>ul</code>	Lista no ordenada.
<code>ol</code>	Lista ordenada. <code>type</code> puede ser <code>1</code> , <code>A</code> , <code>a</code> , <code>I</code> , <code>i</code> ; <code>start</code> indica el número inicial.
<code>li</code>	Elemento de <code>ul</code> u <code>ol</code> .
<code>dl</code>	Lista de definición.
<code>dt</code>	Término a definir.
<code>dd</code>	Definición del término.

Imágenes

`img` define un espacio para contener imágenes. Sus atributos clave son:

- `src`: ruta de la imagen; puede ser absoluta o relativa;
- `alt`: texto alternativo si no carga o para lectores de pantalla;
- `width` y `height`: tamaño de la imagen;
- `style`: alternativa para controlar tamaño mediante estilos;
- `loading`: `eager` carga inmediatamente y `lazy` retrasa la carga hasta que la imagen sea necesaria.

Formatos citados:

Formato	Tipo	Uso recomendado
JPEG/JPG	Con pérdida	Fotografías.
PNG	Sin pérdida	Transparencia alta.
GIF	Sin pérdida + animación	256 colores; compatibilidad antigua.
SVG	Vectorial	Iconos y gráficos escalables.
ICO	Bitmap	Favicons.
BMP	Bitmap	Soportado, pero poco eficiente.
WebP	Compresión con/sin pérdida, alfa y animación	Logos, UI, iconos simples y animaciones.
AVIF	Compresión superior a WebP, alfa y animación	Fotos, animaciones y mejor relación calidad/tamaño.

Notas:

- Para fotos: **AVIF**.
- Para logos, UI e iconos simples: **WebP o PNG**.
- Para animaciones: **WebP o AVIF**; GIF solo si se necesita compatibilidad total antigua.
- Para compatibilidad total con dispositivos antiguos: **JPEG + PNG**.
- **AOMedia** es un consorcio tecnológico formado por empresas como Google, Apple, Mozilla, Netflix, Amazon, Microsoft y Cisco, entre otras.

Mapa de imagen y `picture`

Etiqueta/atributo	Uso
<code>map</code>	Define un mapa de imagen; se identifica con <code>name</code> .
<code>area</code>	Define zonas clicables dentro del mapa.
<code>shape</code>	Forma del área: <code>rect</code> , <code>circle</code> , <code>poly</code> o <code>default</code> .
<code>coords</code>	Coordenadas de la zona.

Etiqueta/atributo	Uso
<code>usemap="#nombre"</code>	Relaciona la imagen con el mapa.
<code>picture</code>	Permite mostrar imágenes diferentes según dispositivo o tamaño.
<code>source</code>	Dentro de <code>picture</code> , aporta alternativas con <code>srcset</code> y condiciones <code>media</code> .

Tablas

Etiqueta/atributo	Uso
<code>table</code>	Tabla de datos; no debe usarse para maquetar.
<code>tr</code>	Fila.
<code>td</code>	Celda de datos.
<code>th</code>	Celda de cabecera; al principio de <code>tr</code> crea cabeceras de fila.
<code>caption</code>	Título o contexto de la tabla.
<code>colgroup</code>	Conjunto de grupos de columnas.
<code>col</code>	Grupo de columnas; <code>span</code> indica cuántas columnas abarca.
<code>thead</code>	Cabecera de tabla.
<code>tbody</code>	Cuerpo de tabla.
<code>tfoot</code>	Pie de tabla.
<code>colspan</code>	Número de columnas que abarca una celda.
<code>rowspan</code>	Número de filas que abarca una celda.

2.7 HTML5: simplificación, multimedia y nuevas etiquetas

HTML5 simplifica la sintaxis:

- doctype: `<!DOCTYPE html>;`
- codificación: `<meta charset="utf-8">;`
- más flexibilidad en mayúsculas/minúsculas y comillas: `id="daw"`, `id=daw`, `ID="daw"`;
- nuevas etiquetas semánticas: `header`, `footer`, `section`, `article`, `aside`, `nav`, `figure`, `figcaption`;
- más recursos nativos: `audio`, `video`, `track`, `canvas`, `iframe`;
- nuevos atributos, nuevos atributos de formulario, APIs nativas, menos necesidad de plugins y mejor acceso al DOM desde JavaScript.

Multimedia

Etiqueta	Uso
<code>audio</code>	Inserta sonido reproducible.
<code>video</code>	Inserta vídeo reproducible.
<code>source</code>	Ofrece varias fuentes del mismo recurso para mejorar compatibilidad.
<code>track</code>	Añade subtítulos o pistas de texto, importante para accesibilidad.
<code>canvas</code>	Superficie de dibujo controlada con JavaScript.
<code>iframe</code>	Incrusta otra página o recurso externo.

La etiqueta `video` admite:

- `controls`: controles de reproducción, pausa y volumen;
- `autoplay`: arranque automático al cargar;
- `muted`: sonido silenciado al arrancar.

Formatos de vídeo citados:

Formato/códec	Soporte indicado
MP4 / H.264	Muy alto.
WebM / VP8-VP9	Alto.
Ogg / Theora	Bueno.
MP4 / H.265	Parcial.
AV1	Amplio pero parcial.
AVI, WMV, FLV	Nulo en HTML5 moderno.

Otras etiquetas HTML5

Etiqueta	Uso
<code>details</code>	Bloque desplegable que el usuario puede abrir/cerrar.
<code>summary</code>	Texto visible que controla <code>details</code> .
<code>mark</code>	Resalta texto relevante.
<code>meter</code>	Medida dentro de un rango conocido.
<code>time</code>	Semántica para fechas y horas.
<code>progress</code>	Progreso de una tarea.

2.8 Formularios

Los formularios HTML permiten aplicaciones interactivas al recopilar información del usuario.

Tienen dos partes:

- **acción con la información:** etiqueta `form`;
- **estructura de entrada:** `input`, `textarea`, `select`, `datalist`, `fieldset`, etc.

```
<form action="#" method="post">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" required>

  <br><br>

  <label for="email">Correo electrónico:</label>
  <input type="email" id="email" name="email" required>

  <br><br>

  <label for="mensaje">Mensaje:</label>
  <textarea id="mensaje" name="mensaje" rows="4"></textarea>

  <br><br>

  <button type="submit">Enviar</button>
</form>
```

form

`form` define un formulario y funciona como contenedor de controles.

Atributo	Uso
<code>name</code>	Nombre del formulario.
<code>action</code>	Recurso o programa del servidor que recibe los datos.
<code>method</code>	Método HTTP usado para enviar datos: <code>get</code> o <code>post</code> .
<code>autocomplete</code>	Activa o desactiva autocompletado.
<code>novalidate</code>	Evita la validación al enviar.

`GET` es el método por defecto: añade datos a la URL en pares nombre/valor, tiene límite práctico de longitud, no debe usarse para datos sensibles y permite guardar el resultado como marcador. `POST`

coloca los datos en la petición HTTP, no tiene la misma limitación de longitud y no permite guardar el resultado como marcador.

Controles `input`

`input` crea elementos gráficos para capturar información.

Reglas generales:

- `name` es obligatorio si se quiere enviar el valor al servidor;
- `value` almacena el valor inicial;
- `type` determina el control;
- algunos controles tienen eventos asociados como `onBlur`, `onFocus`, `onClick`, `onChange`, `onSelect`, `onKeyDown`, `onKeyPress` y `onKeyUp`.

type	Uso	Eventos citados
button	Botón estándar para acción específica, normalmente JavaScript.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
text	Texto breve en una línea: nombres, títulos, búsquedas o usuario.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code> , <code>onChange</code> , <code>onSelect</code>
password	Texto no legible visualmente para accesos o autenticación.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code> , <code>onChange</code> , <code>onSelect</code>
hidden	Campo oculto con datos útiles para el servidor, como identificadores o tokens.	-
submit	Botón de envío.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
reset	Restaura valores iniciales del formulario.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
file	Selección de archivo local para subir documentos, imágenes o adjuntos.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code> , <code>onChange</code>
checkbox	Selección múltiple.	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
radio	Selección exclusiva; varias opciones comparten <code>name</code> .	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
image	Botón de imagen; envía coordenadas <code>coord.x</code> y <code>coord.y</code> .	<code>onBlur</code> , <code>onFocus</code> , <code>onClick</code>
range	Selector gráfico dentro de un rango: edad, volumen, porcentaje o escala.	-
date	Selección de día: cumpleaños, reservas o citas.	-
week	Selección de semana: planificación o turnos.	-
month	Selección de mes: informes, periodos o filtros.	-
color	Paleta de selección de color para temas o personalización.	-

Ejemplos:

Otros controles de formulario

Etiqueta	Uso
<code>textarea</code>	Texto multilínea; eventos <code>onKeyDown</code> , <code>onKeyPress</code> , <code>onKeyUp</code> .
<code>select</code>	Lista desplegable seleccionable con teclado o ratón; <code>multiple</code> permite varias opciones.
<code>option</code>	Opción de <code>select</code> o <code>datalist</code> ; <code>selected</code> marca una opción preseleccionada.
<code>datalist</code>	Lista de valores sugeridos para un <code>input</code> ; el <code>input</code> la referencia mediante <code>list</code> .
<code>fieldset</code>	Agrupar controles relacionados.
<code>legend</code>	Título de un <code>fieldset</code> .
<code>label</code>	Texto asociado a un control; <code>for</code> debe coincidir con el <code>id</code> del <code>input</code> .

3. CSS y Diseño Responsivo

3.1 Qué es CSS

CSS significa **Cascading Style Sheets**. Fue introducido en HTML 4.0 y propuesto por el W3C para separar **contenido** y **formato**:

- HTML describe la estructura y el contenido.
- CSS describe cómo se muestran los elementos HTML.
- El navegador carga HTML, aplica las hojas de estilo y pinta el resultado.

CSS controla color, tipografía, márgenes, tamaños, fondos, distribución y colocación de elementos. Su origen es asociado a **Hakon Wium Lie**.

Una regla de estilo tiene esta forma:

```
selector { propiedad: valor; propiedad: valor; }
```

Parte	Función
Selector	Determina qué elementos se ven afectados.
Declaración	Uno o más pares <code>propiedad: valor</code> , separados por <code>;</code> .

Los comentarios se escriben con `/* comentario */`; son multilínea y pueden aparecer dentro de una línea.

3.2 Cómo añadir CSS, cascada y precedencia

Formas de aplicar CSS

Forma	Uso
En línea	Usa el atributo <code>style</code> en un elemento concreto. Es puntual y poco reutilizable.
Hoja interna	Usa <code><style></code> dentro de <code>head</code> ; afecta a la página actual.
Hoja externa desde HTML	Usa <code><link rel="stylesheet" href="hoja.css"></code> ; es la forma normal de trabajo.
Hoja externa desde CSS	Usa <code>@import</code> ; sirve para modularizar hojas de estilo.

CSS también permite cargar hojas según el contexto:

Caso	Ejemplo o idea
Según dispositivo	<code><link rel="stylesheet" media="screen" href="hoja.css"></code> ; valores como <code>screen</code> , <code>print</code> , <code>handheld</code> .
Estilos de sitio	Hoja común, por ejemplo <code>sitio.css</code> .
Estilos de página	Hoja específica, por ejemplo <code>pagina.css</code> .
Estilos alternativos	En Firefox se cita <code>rel="alternate stylesheet"</code> con <code>title="nombre"</code> .

Cascada y precedencia

Cuando varias reglas afectan a un mismo elemento, CSS decide con:

1. origen e importancia;
2. especificidad;
3. orden de aparición.

Capas principales de la cascada:

Navegador -> Usuario -> Autor -> Autor + !important -> Usuario + !important -> Navegador + !important

Dentro de los estilos de autor, la precedencia práctica es:

Criterio	Idea
Estilo en línea	Tiene prioridad sobre reglas internas o externas normales.
Especificidad	Gana el selector más específico: primero identificadores, luego clases/atributos/pseudoclases, después etiquetas/pseudoelementos.
Última regla	Si la especificidad empata, gana la regla que aparece más tarde.

Valores especiales de propiedades:

Valor	Uso
<code>initial</code>	Recupera el valor inicial de la propiedad.
<code>auto</code>	El navegador calcula el valor.
<code>inherit</code>	Toma el valor heredado del elemento padre.
<code>unset</code>	Se comporta como heredado o inicial según la propiedad.
<code>revert</code>	Intenta volver al valor de una capa anterior de la cascada.

Unidades de medida

Unidad	Significado
%	Relativa al tamaño del contenedor padre.
vw / vh	1% del ancho/alto del viewport.
em	Relativa al tamaño de fuente del padre o contexto.
rem	Relativa al tamaño de fuente raíz (<code>html</code>).
fit-content	Ajuste al tamaño exacto del contenido.
px, cm, mm	Unidades absolutas o físicas usadas según contexto.

3.3 Selectores, pseudoclasas y herencia

Los selectores CSS localizan elementos HTML a los que se quiere aplicar estilo.

Selectores básicos

Selector	Regla	Elemento afectado
Universal	<code>* {}</code>	Todos los elementos.
Agrupación	<code>h1, h2, h3 {}</code>	Varios selectores comparten regla.
Por etiqueta	<code>h1 {}</code>	Todos los <code>h1</code> .
Por clase	<code>.header {}</code>	Elementos con <code>class="header"</code> .
Por identificador	<code>#principal {}</code>	Elemento con <code>id="principal"</code> .
Por atributo	<code>a[href*="usc"] {}</code>	Enlaces cuyo <code>href</code> contiene <code>usc</code> .

Reglas prácticas:

- `id` debe ser único en el documento.
- `class` permite reutilización y combinación, por ejemplo `class="clase1 clase2"`.
- La combinación `etiqueta.clase1.clase2` selecciona etiquetas que tengan todas esas clases.

Combinadores

Combinador	Significado	Ejemplo
Espacio	Descendiente, aunque haya niveles intermedios.	<code>div a {}</code>
<code>></code>	Hijo directo.	<code>div > a {}</code>
<code>+</code>	Hermano adyacente situado a la derecha.	<code>div + a {}</code>
<code>~</code>	Hermanos posteriores situados a la derecha.	<code>div ~ a {}</code>

Pseudoclases y pseudoelementos

Tipo	Casos citados
Pseudoclases interactivas	<code>:link</code> , <code>:visited</code> , <code>:active</code> , <code>:hover</code> , <code>:focus</code> .
Pseudoclases estructurales	<code>:first-child</code> , <code>:last-child</code> , <code>:nth-child(n)</code> , <code>:lang()</code> .
Pseudoelementos de texto	<code>::first-line</code> , <code>::first-letter</code> .
Pseudoelementos de contenido	<code>::before</code> , <code>::after</code> , <code>::marker</code> , <code>::selection</code> , <code>::backdrop</code> .

Una pseudoclase define el estilo de un **estado** del elemento (`selector:pseudoclase`). Un pseudoelemento define el estilo de una **parte concreta** del elemento (`selector::pseudoelemento`).

Herencia

Algunas propiedades se heredan y otras no.

- Suelen heredarse: `color`, tipografía y propiedades de texto.
- No suelen heredarse: `margin`, `padding`, `border`, `background`, dimensiones y posicionamiento.
- Puede forzarse con `inherit`.

Idea clave: la herencia importa mucho en tipografía y menos en layout.

3.4 Modelo de caja y propiedades base

Todo elemento HTML se representa como una caja:

Parte	Función
Content	Contenido real: texto, imágenes, etc.
Padding	Área transparente alrededor del contenido.
Border	Borde que rodea contenido y padding.
Margin	Separación exterior respecto a otros elementos.
Outline	Línea externa al borde, útil para foco o resaltado.

El ancho total de la caja se calcula como:

```
width + padding-left + padding-right + border-left + border-right
```

El alto total se calcula como:

```
height + padding-top + padding-bottom + border-top + border-bottom
```

El margen afecta al espacio total ocupado, pero no forma parte del tamaño de la caja.

box-sizing

Valor	Efecto
content-box	width y height solo miden el contenido; padding y border se suman aparte.
border-box	width y height incluyen contenido, padding y border. Suele ser más predecible.

Propiedades de caja

Propiedad	Uso
width, height	Tamaño del elemento; auto deja calcular al navegador.
min-width, max-width	Evitan tamaños demasiado pequeños o grandes; ayudan a impedir barras de desplazamiento innecesarias.
min-height, max-height	Equivalente vertical para altura.
padding	Resumen de padding-top/right/bottom/left.
margin	Resumen de margin-top/right/bottom/left; los márgenes verticales adyacentes pueden colapsar y quedarse en el mayor.
border-style	dotted, dashed, solid, double, groove, ridge, inset, outset, none, hidden.
border-width	thin, medium, thick o medidas como Xpx.
border-color	Color del borde.
border-radius	Bordes redondeados.
outline-style	Mismos estilos generales que border-style.
outline-width	Ancho del outline.
outline-color	Color del outline.
outline-offset	Distancia entre outline y borde; puede ser negativa.

Las propiedades resumen de lados admiten:

- 1 valor: todos los lados;
- 2 valores: vertical y horizontal;
- 3 valores: superior, horizontal e inferior;
- 4 valores: superior, derecho, inferior e izquierdo.

Fondo, desbordamiento, color y visibilidad

Propiedad	Uso
<code>background-color</code>	Color de fondo.
<code>background-image: url(...)</code>	Imagen de fondo.
<code>background-repeat</code>	<code>repeat</code> , <code>repeat-x</code> , <code>repeat-y</code> , <code>no-repeat</code> .
<code>background-position</code>	Posición inicial: <code>left top</code> , porcentajes, coordenadas, etc.
<code>background-attachment</code>	<code>scroll</code> , <code>fixed</code> , <code>local</code> ; controla si el fondo se mueve con la página o con el contenido.
<code>background</code>	Propiedad resumen de las anteriores.
<code>overflow</code>	<code>visible</code> , <code>hidden</code> , <code>scroll</code> , <code>auto</code> ; también existen <code>overflow-x</code> y <code>overflow-y</code> .
<code>border-collapse</code>	En tablas: <code>separate</code> o <code>collapse</code> ; si se separan puede usarse <code>border-spacing</code> .
<code>opacity</code>	Transparencia entre <code>0.0</code> y <code>1.0</code> .
<code>z-index</code>	Orden de apilado; solo funciona en elementos posicionados. Si no se indica, manda el orden del HTML.
<code>visibility</code>	<code>visible</code> , <code>hidden</code> , <code>collapse</code> ; <code>hidden</code> oculta pero conserva espacio.

Formas de indicar color RGB:

- `#rgb`;
- `#rrggbb`;
- `rgb(r, g, b)` y `rgba(r, g, b, a)`;
- `rgb(r%, g%, b%)` y variantes con alfa;
- colores predefinidos.

3.5 `display`, posicionamiento y layout clásico

`div`, `span`, `display` y `visibility`

`div` es un contenedor genérico de bloque; `span` es un contenedor genérico en línea. No aportan significado semántico, pero sirven para agrupar y aplicar estilo.

Propiedad/valor	Uso
<code>display: none</code>	Elimina el elemento visualmente y también el espacio que ocupaba.
<code>display: inline</code>	Hace que el elemento se comporte como elemento de línea.
<code>display: block</code>	Hace que el elemento se comporte como bloque.

Propiedad/valor	Uso
<code>display: inline-block</code>	Se coloca en línea, pero conserva control de <code>width</code> y <code>height</code> .
<code>visibility: hidden</code>	Oculto el elemento, pero mantiene su espacio.
<code>visibility: collapse</code>	Elimina fila/columna/grupo de tabla o ítem flex como si fuera <code>display: none</code> .

Posicionamiento

Por defecto, el navegador distribuye elementos siguiendo el flujo natural del fichero fuente.

Valor de <code>position</code>	Comportamiento
<code>static</code>	Valor por defecto; sigue el flujo normal.
<code>relative</code>	Permanece en el flujo, pero puede desplazarse respecto a su posición natural.
<code>absolute</code>	Sale del flujo y se coloca respecto a su contenedor padre posicionado más cercano; se resume como relativo al contenedor padre.
<code>fixed</code>	Sale del flujo y se coloca respecto a la ventana del navegador.
<code>sticky</code>	Se comporta como <code>relative</code> hasta cierto scroll y luego como <code>fixed</code> .

NOTA

El tío te chanta esto y se queda tan ancho, es jodido expresar un concepto tan mal que locura.

Un elemento puede ser **contenedor** de otros elementos aunque no tenga `position` definido.

Pero para que un hijo con `position: absolute` lo tome como **referencia de posicionamiento**, el contenedor debe estar **posicionado**, es decir, tener un `position` distinto de `static`.

Ejemplo habitual:

```
.contenedor {  
  position: relative;  
}  
  
.hijo {  
  position: absolute;  
  top: 0;  
  left: 0;  
}
```

Si el contenedor no tiene `position: relative`, `absolute`, `fixed` o `sticky`, el hijo buscará otro ancestro posicionado. Si no encuentra ninguno, se posicionará respecto al bloque inicial de la página.

La localización se especifica con `top`, `right`, `bottom` y `left`.

`float` y `clear`

Propiedad	Uso
<code>float</code>	Hace flotar un elemento a <code>left</code> , <code>right</code> o <code>none</code> , elevándolo sobre la página.
<code>clear</code>	Controla qué ocurre con elementos colocados junto a flotantes: <code>left</code> , <code>right</code> , <code>both</code> , <code>none</code> .

`float` desplaza elementos de bloque cercanos para ocupar el hueco que deja; los elementos de línea lo rodean. Si se combina con `position: relative`, parte de una posición relativa. Hoy se usa sobre todo para texto alrededor de imágenes o mantenimiento de layouts antiguos.

```
clear: left; /* baja si hay un float a la izquierda */  
clear: right; /* baja si hay un float a la derecha */
```

```
clear: both; /* baja si hay floats a cualquier lado */
clear: none; /* valor normal */
```

CSS3 y prefijos de navegador

Algunos efectos de CSS3 dependían históricamente del navegador y podían necesitar prefijos:

- `-moz-`: Firefox;
- `-o-`: Opera;
- `-webkit-`: Safari y Chrome;
- `-ms-`: Internet Explorer.

3.6 Texto y tipografía

Las fuentes se agrupan en familias:

Familia	Rasgo
<code>sans-serif</code>	Líneas limpias.
<code>serif</code>	Serifas en vértices.
<code>monospace</code>	Todas las letras con el mismo ancho.
<code>cursive</code>	Imitan escritura a mano.
<code>fantasy</code>	Decorativas.

`font-family` debe listar varias fuentes: primero la deseada y al final una familia genérica. Los nombres compuestos se colocan entre comillas. Para fuentes externas se usa `@font-face`, indicando el nombre y la URL de la fuente.

Cualquier símbolo no presente en el teclado puede obtenerse mediante Unicode: `&#numero` o `&#xnumero`.

Propiedad	Uso
<code>font-size</code>	Tamaño: <code>xx-small</code> , <code>small</code> , <code>medium</code> , <code>large</code> , <code>Xpx</code> , <code>Xrem</code> , <code>Xvw</code> , etc.
<code>font-weight</code>	Grosor: <code>normal</code> , <code>bold</code> , <code>bolder</code> , <code>lighter</code> , <code>100</code> a <code>900</code> .
<code>font-style</code>	<code>normal</code> , <code>italic</code> , <code>oblique</code> .
<code>font</code>	Resumen de <code>font-style</code> , <code>font-variant</code> , <code>font-weight</code> , <code>font-stretch</code> , <code>font-size</code> , <code>line-height</code> , <code>font-family</code> .
<code>color</code>	Color del texto.
<code>text-align</code>	Alineación horizontal: <code>left</code> , <code>right</code> , <code>center</code> , <code>justify</code> .
<code>text-align-last</code>	Alineación horizontal de la última línea.

Propiedad	Uso
<code>vertical-align</code>	Alineación vertical: <code>baseline</code> , medidas, <code>sub</code> , <code>super</code> , <code>top</code> , etc.
<code>text-decoration</code>	Resumen de línea, color y estilo: <code>none</code> , <code>underline</code> , <code>overline</code> , <code>line-through</code> ; <code>blink</code> está obsoleto.
<code>text-transform</code>	<code>none</code> , <code>capitalize</code> , <code>uppercase</code> , <code>lowercase</code> .
<code>text-indent</code>	Sangría de la primera línea.
<code>letter-spacing</code>	Espacio entre caracteres; en el PDF aparece como <code>text-spacing</code> .
<code>word-spacing</code>	Espacio entre palabras.
<code>line-height</code>	Espacio entre líneas.
<code>white-space</code>	<code>normal</code> , <code>nowrap</code> , <code>pre</code> , <code>pre-line</code> , <code>pre-wrap</code> .
<code>text-shadow</code>	Sombras: <code>x y blur color</code> , con posibilidad de varias sombras.
<code>direction</code>	Dirección de escritura: <code>rtl</code> o <code>ltr</code> .

La tipografía afecta a la apariencia y al espacio que ocupa el contenido, por lo que también influye en el layout.

3.7 Diseño responsivo

El **layout** es la forma en la que se disponen los elementos de una página.

Enfoque	Idea
Layout fijo	Tamaño absoluto de cajas; si cambia la pantalla aparecen barras o espacios desaprovechados.
Layout líquido	El navegador adapta tamaños al ancho disponible y a la cantidad de contenido.
Diseño responsivo	La página se adapta automáticamente al tamaño del dispositivo.

La aproximación actual es **mobile first**: primero se diseña una versión simple para móviles, normalmente a una columna, y después se amplía a varias columnas en pantallas mayores.

Hitos:

- **Cameron Adams (2004)**: diseños que se adaptan al tamaño de pantalla usando JavaScript para detectar tamaño y cargar CSS adecuado.
- **Ethan Marcotte (2010)**: término "diseño responsivo", basado en rejillas fluidas, imágenes fluidas y solicitudes de tipo de medio.

Viewport

El **viewport** es el área visible de una página web para el usuario. La metaetiqueta habitual establece el ancho al dispositivo y el zoom inicial a 1:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Reglas @media

Las reglas @media aplican CSS solo si se cumple una condición:

```
@media not/only mediatype and (mediafeature) { CSS-Code; }
```

Parte	Valores o uso
mediatype	all por defecto, print, screen.
mediafeature	width, height, orientation (portrait o landscape), resolution, entre otras.
not	Invierte la condición.
only	Evita problemas con navegadores antiguos; no tiene efecto práctico en navegadores modernos.
and	Combina tipo de medio y características.

También pueden incluirse condiciones de medios directamente en etiquetas link para seleccionar hojas alternativas.

Puntos de ruptura

Los **breakpoints** son dimensiones a partir de las cuales se planifica un cambio de diseño.

Punto	Dispositivo orientativo
min-width: 320px	Móviles pequeños.
min-width: 480px	Pequeños dispositivos y la mayoría de móviles.
min-width: 768px	La mayoría de tablets.
min-width: 992px	Ordenadores con pantallas pequeñas.
min-width: 1200px	Grandes dispositivos y pantallas grandes.

Tipografía e imágenes responsivas:

- La tipografía responsiva adapta tamaños al dispositivo.
- Las imágenes responsivas adaptan el recurso o su tamaño al dispositivo; la solución actual se apoya en imágenes preparadas para distintos tamaños/orígenes.

3.8 Técnicas de layout responsivo

Flexible grids tradicionales

Mecanismo antiguo de responsividad basado en:

- reglas `@media`;
- `float`;
- abuso de medidas relativas en `%`;
- cálculo `width = 100 * ancho_deseado / ancho_pagina`.

Funcionaba, pero era más frágil y menos mantenible que Flexbox o Grid.

CSS Multicol

Multicol divide un contenedor en columnas y deja que el navegador calcule el reparto según el viewport.

Propiedad	Uso
<code>column-count</code>	Número de columnas del contenedor.

Es útil para texto continuo, artículos o diseños tipo periódico, pero no ofrece control completo de layout.

Flexbox

Flexbox facilita la disposición flexible en **una dimensión**: filas o columnas.

Una flexbox tiene:

- un **contenedor flex** con `display: flex`;
- varios **ítems flex**, que son sus hijos directos.

Propiedades del contenedor:

Propiedad	Valores o uso
<code>flex-direction</code>	<code>row</code> , <code>column</code> , <code>row-reverse</code> , <code>column-reverse</code> .
<code>flex-wrap</code>	<code>nowrap</code> , <code>wrap</code> , <code>wrap-reverse</code> .
<code>flex-flow</code>	Resumen de <code>flex-direction</code> y <code>flex-wrap</code> .
<code>align-items</code>	Alinea ítems en el eje secundario: <code>flex-start</code> , <code>flex-end</code> , <code>stretch</code> , <code>baseline</code> .
<code>align-content</code>	Alinea líneas cuando hay varias por <code>wrap</code> : <code>stretch</code> , <code>center</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code> .

Propiedad	Valores o uso
<code>justify-content</code>	Reparte ítems en el eje principal: <code>flex-start</code> , <code>flex-end</code> , <code>center</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code> .
<code>place-content</code>	Resumen de alineación de contenido.
<code>gap</code>	Resumen de <code>row-gap</code> y <code>column-gap</code> .

Propiedades de los ítems:

Propiedad	Uso
<code>flex-grow</code>	Cuánto crece el ítem respecto a los demás.
<code>flex-shrink</code>	Cuánto decrece el ítem respecto a los demás.
<code>flex-basis</code>	Longitud inicial del ítem.
<code>flex</code>	Resumen de <code>flex-grow</code> , <code>flex-shrink</code> y <code>flex-basis</code> .
<code>order</code>	Orden visual dentro del contenedor.
<code>align-self</code>	Alineación individual; sobrescribe <code>align-items</code> .

Notas del apunte:

- `nowrap` es el valor por defecto y puede hacer que los ítems encojan.
- `wrap` permite que bajen a otra línea.
- `flex-shrink: 0` impide que un ítem se encoja, aunque pueda salirse del contenedor.

CSS Grid

Grid facilita layouts flexibles en **dos dimensiones**: filas y columnas.

Una grid tiene:

- un **contenedor grid** con `display: grid`;
- uno o más **ítems grid**, hijos directos del contenedor.

Propiedades del contenedor:

Propiedad	Uso
<code>grid-template-rows</code>	Tamaño de filas.
<code>grid-template-columns</code>	Tamaño y número de columnas; admite <code>px</code> , <code>%</code> , <code>auto</code> y <code>fr</code> .

Propiedad	Uso
<code>grid-template-areas</code>	Define áreas con nombres; cada fila va entre comillas y <code>.</code> indica celda vacía.
<code>grid-template</code>	Resumen de las tres anteriores.
<code>align-items</code>	Alineamiento vertical por defecto: <code>stretch</code> , <code>start</code> , <code>end</code> , <code>center</code> , <code>baseline</code> .
<code>justify-items</code>	Alineamiento horizontal por defecto: <code>stretch</code> , <code>start</code> , <code>end</code> , <code>center</code> .
<code>align-content</code>	Alinea la rejilla verticalmente: <code>start</code> , <code>end</code> , <code>center</code> , <code>stretch</code> , <code>space-around</code> , <code>space-between</code> , <code>space-evenly</code> .
<code>justify-content</code>	Alinea la rejilla horizontalmente con valores equivalentes.
<code>gap</code>	Resumen de <code>row-gap</code> y <code>column-gap</code> .

Propiedades de los ítems:

Propiedad	Uso
<code>grid-column-start</code> / <code>grid-column-end</code>	Columna inicial y final.
<code>grid-column</code>	Resumen de las dos anteriores.
<code>grid-row-start</code> / <code>grid-row-end</code>	Fila inicial y final.
<code>grid-row</code>	Resumen de las dos anteriores.
<code>grid-area</code>	Nombre del ítem o área.
<code>justify-self</code>	Alineación horizontal individual; sobrescribe <code>justify-items</code> .
<code>align-self</code>	Alineación vertical individual; sobrescribe <code>align-items</code> .

Notas prácticas añadidas:

- `fr` reparte el espacio disponible en fracciones.
- `repeat(auto-fit, minmax(220px, 1fr))` es un patrón útil para galerías responsivas.
- Si el problema es una fila o columna, suele encajar Flexbox; si son filas y columnas a la vez, suele encajar Grid.

3.9 Frameworks CSS y Bootstrap

Un **framework CSS** es una biblioteca de reglas predefinidas apoyadas en clases HTML.

Ventajas	Inconvenientes
Facilitan compatibilidad entre navegadores.	Importan código innecesario y aumentan ancho de banda.
Facilitan responsividad entre dispositivos.	Reducen parte del control sobre el diseño.
Simplifican el desarrollo.	Limitan opciones visuales si se usan sin personalizar.
Aportan fiabilidad y eficacia.	Pueden homogeneizar demasiado la interfaz.

Bootstrap

Bootstrap es una librería orientada al RWD, especialmente a móviles. Fue creada por **Mark Otto** y **Jacob Thornton** en Twitter y liberada en GitHub en 2011.

Según el material:

- la versión usada actualmente en clase es Bootstrap 5, publicada en 2021;
- Bootstrap 5 aporta componentes nuevos, hoja de estilos más rápida y mayor capacidad de respuesta;
- es compatible con versiones estables recientes de navegadores y plataformas;
- no es compatible con Internet Explorer 11 y anteriores;
- cambió a JavaScript nativo en lugar de depender de jQuery.

Para integrarlo se añade la hoja CSS y, si se usan componentes interactivos, también el bundle JavaScript.

Contenedores, espaciado y layout

Clase	Uso
<code>.container</code>	Contenedor con <code>max-width</code> adaptado a puntos de ruptura prefijados.
<code>.container-XX</code>	Variante según sufijo <code>sm</code> , <code>md</code> , <code>lg</code> , <code>xl</code> , <code>xxl</code> .
<code>.container-fluid</code>	Contenedor líquido con <code>max-width: 100%</code> en cualquier punto.

Espaciados:

`{propiedad}{lado}-{tamaño}` para `xs` y `{propiedad}{lado}-{punto-ruptura}-{tamaño}` para `sm`, `md`, `lg`, `xl`, `xxl`.

Parte	Valores
Propiedad	<code>m</code> para <code>margin</code> , <code>p</code> para <code>padding</code> .
Lado	<code>t</code> , <code>b</code> , <code>l</code> , <code>r</code> , <code>x</code> para izquierda/derecha, <code>y</code> para superior/inferior.
Tamaño	<code>0</code> , <code>1</code> , <code>2</code> , <code>3</code> , <code>4</code> , <code>5</code> , <code>auto</code> ; se basan en <code>\$spacer</code> .

El layout de Bootstrap se basa en Flexbox y en una rejilla de hasta **12 columnas**:

1. crear un contenedor con `.container` o `.container-fluid`;
2. crear filas con `.row`;
3. definir columnas con `col-{dispositivo}-{numero}`.

Propiedades y componentes fijados o facilitados por Bootstrap según el PDF:

- texto/tipografía: `rem = 16px`, `$spacer = 1rem`, `line-height = 1.5rem`;
- formatos de tablas;
- imágenes con redondeo de esquinas y marcos;
- párrafos resaltados mediante `jumbotron`;
- cajas con bordes redondeados mediante `well`;
- iconos con `glyphicon`;
- paginación y migas de pan con `pagination` y `breadcrumb`;
- desplegados con `dropdown`;
- barras de navegación con `navbar` y `navbar-*`;
- carruseles con `carousel-*`.

3.10 Validación y compatibilidad

Para validar CSS se puede usar:

`http://jigsaw.w3.org/css-validator`

También conviene recordar que muchas propiedades modernas ya no requieren prefijo en navegadores actuales, pero los prefijos históricos pueden aparecer en código antiguo, documentación o ejemplos heredados.

4. JavaScript

4.1 Introducción e integración con HTML

JavaScript (JS) es un lenguaje de programación **interpretado** creado en **1995** por **Brendan Eich** para añadir interactividad a **Netscape Navigator 2.0**.

Datos históricos:

- nombre original: **Mocha**;
- después se rebautiza como **JavaScript** mediante acuerdo con Sun Microsystems;
- **no es Java**;
- en **1997** comienza su estandarización en **ECMA** como **ECMAScript**;
- posteriormente también se convierte en estándar **ISO**;
- con JS aparece el concepto de **DHTML** o HTML dinámico.

JS normalmente se ejecuta en el **lado cliente** dentro del navegador, aunque hoy también puede ejecutarse en servidor mediante **Node.js**; ese uso no forma parte del tema.

Integración con HTML

Se incluye mediante la etiqueta `script`:

Forma	Uso
Código interno	El código va como contenido de <code><script></code> .
Fichero externo	Se enlaza con <code><script src="codigo.js"></script></code> .

Para mejorar la carga de la página se aconseja colocar el script al final, antes de `</body>`, de modo que el HTML principal ya esté creado cuando JS lo manipule.

Microsoft desarrolló dos versiones propias para Internet Explorer:

- **VBScript**: sintaxis basada en Visual Basic e incompatible con otros navegadores.
- **JScript**: versión similar a ECMAScript, pero con diferencias en el modelo de objetos.

Las incompatibilidades entre JScript y JavaScript llevaron al W3C a definir el **HTML DOM** como criterio común.

4.2 HTML DOM y APIs del navegador

El **DOM (Document Object Model)** es una API creada por W3C para acceder de forma estándar a documentos **HTML**, **XML** y **SVG**, permitiendo modificar contenido, estructura y estilo.

Parte	Descripción
Core DOM	Modelo estándar para cualquier documento estructurado.
XML DOM	Modelo estándar para documentos XML.
HTML DOM	Modelo estándar para documentos HTML.

Objetos e interfaces principales

`document` representa la página mediante el árbol DOM. Es una propiedad de `window`, que representa el navegador, y una instancia de `HTMLDocument`.

Objeto/interfaz	Uso
<code>window</code>	Objeto global de la ventana o pestaña; contiene <code>document</code> .
<code>document</code> / <code>HTMLDocument</code>	Representa el documento HTML completo.
<code>Element</code> / <code>HTMLElement</code>	Representa una etiqueta del documento.
<code>Attr</code>	Representa atributos; propiedades como <code>name</code> , <code>value</code> , <code>specified</code> .
<code>Text</code>	Nodo de texto; propiedades como <code>data</code> , <code>length</code> ; métodos como <code>appendData()</code> , <code>deleteData()</code> , <code>insertData()</code> , <code>replaceData()</code> .
<code>Comment</code>	Comentarios del documento.
<code>Style</code> / <code>CSSStyleDeclaration</code>	Estilos asociados a etiquetas; <code>cssText</code> , <code>length</code> , <code>parentRule</code> , <code>getPropertyValue()</code> , <code>setProperty()</code> , etc.
<code>Event</code>	Eventos asociados a etiquetas.
<code>location</code>	URL actual y redirecciones.
<code>history</code>	Historial de navegación.
<code>navigator</code>	Datos del navegador/dispositivo.
<code>screen</code>	Datos de pantalla.
<code>frames</code>	Acceso a marcos o <code>iframe</code> .

Árbol DOM

Una página HTML está formada por etiquetas anidadas que el navegador transforma en un árbol:

- cada etiqueta crea un **nodo elemento**;
- el texto dentro de una etiqueta crea un **nodo de texto**;
- las etiquetas hijas se convierten en nodos hijos del nodo padre.

Con el DOM, JavaScript puede cambiar etiquetas, atributos, estilos, eliminar o añadir nodos, reaccionar a eventos y crear eventos nuevos.

Selección y contenido

Método	Devuelve
<code>document.getElementById("id")</code>	Elemento cuyo <code>id</code> coincide.
<code>document.getElementsByClassName("nombre")</code>	<code>HTMLCollection</code> con elementos que tienen esa clase.
<code>document.getElementsByName("nombre")</code>	<code>HTMLCollection</code> con elementos cuyo <code>name</code> coincide.
<code>document.getElementsByTagName("tag")</code>	<code>HTMLCollection</code> con elementos de esa etiqueta.
<code>document.querySelector("selectorCSS")</code>	Primer elemento que coincide con el selector CSS.
<code>document.querySelectorAll("selectorCSS")</code>	<code>NodeList</code> con todos los elementos que coinciden.

Propiedades de contenido:

Propiedad	Uso
<code>nodeValue</code>	Contenido del nodo actual.
<code>nodeName</code>	Nombre del nodo actual.
<code>nodeType</code>	Tipo de nodo: <code>1</code> elemento, <code>2</code> atributo, <code>3</code> texto, <code>8</code> comentario.
<code>innerHTML</code>	Contenido interpretando HTML.
<code>textContent</code>	Texto del nodo y sus hijos sin interpretar HTML.

Colecciones DOM

Colección	Descripción	Propiedades/métodos
<code>HTMLCollection</code>	Lista de etiquetas HTML; se obtiene con métodos de acceso a etiquetas.	<code>length</code> , <code>item(index)</code> , <code>namedItem(id)</code> .
<code>NodeList</code>	Lista de nodos; puede incluir nodos de texto o atributos.	<code>length</code> , <code>item(index)</code> , <code>entries()</code> , <code>forEach()</code> , <code>keys()</code> , <code>values()</code> .

Colección	Descripción	Propiedades/métodos
<code>NamedNodeMap</code>	Colección de atributos de una etiqueta, accesible con <code>.attributes</code> .	<code>length</code> , <code>getNamedItem()</code> , <code>setNamedItem()</code> , <code>item()</code> , <code>removeNamedItem()</code> .
<code>DOMTokenList</code>	Lista de clases de una etiqueta, accesible con <code>.classList</code> .	<code>length</code> , <code>value</code> , <code>add()</code> , <code>contains()</code> , <code>remove()</code> , <code>replace()</code> , <code>entries()</code> , <code>forEach()</code> , <code>item()</code> , <code>keys()</code> , <code>values()</code> .

Similitudes entre `HTMLCollection` y `NodeList`:

- son listas tipo array;
- tienen `length`;
- permiten acceso por índice.

Diferencias:

- `HTMLCollection` permite acceso por nombre, `id` o índice;
- `NodeList` solo por índice;
- `NodeList` puede contener nodos de atributo y nodos de texto;
- `HTMLCollection` suele ser viva y `NodeList` estática; por eso una puede detectar cambios del DOM automáticamente y la otra no.

Crear, eliminar y recorrer nodos

Método/propiedad	Uso
<code>document.createElement("tag")</code>	Crea un nodo elemento.
<code>document.createTextNode("data")</code>	Crea un nodo de texto.
<code>appendChild(nodo)</code>	Inserta un nodo hijo.
<code>removeChild(nodo)</code>	Elimina un nodo hijo.
<code>replaceChild(nuevo, viejo)</code>	Reemplaza un nodo hijo.
<code>parentNode</code>	Nodo padre.
<code>previousSibling</code> / <code>nextSibling</code>	Hermano anterior o siguiente.
<code>firstChild</code> / <code>lastChild</code>	Primer o último hijo, incluidos nodos de texto.
<code>firstElementChild</code>	Primer hijo que es elemento HTML.
<code>childNodes</code>	Todos los nodos hijos.
<code>children</code>	Solo elementos hijos.

Atributos y estilos

Acceso a atributos:

Método/propiedad	Uso
<code>element.atributo</code>	Acceso directo al valor del atributo.
<code>element.getAttribute("atributo")</code>	Valor del atributo como string.
<code>element.getAttributeNode("atributo")</code>	Nodo <code>Attr</code> del atributo.
<code>element.attributes</code>	<code>NamedNodeMap</code> con los atributos.
<code>element.getAttributeNames()</code>	Array con nombres de atributos.

Manipulación:

Método	Uso
<code>setAttribute("atributo", "valor")</code>	Crea o modifica un atributo.
<code>hasAttribute("atributo")</code>	Comprueba si existe.
<code>removeAttribute("atributo")</code>	Elimina el atributo.

Estilos:

Acceso	Uso
<code>element.style</code>	Acceso al estilo en línea del elemento.
<code>window.getComputedStyle(element, "")</code>	Acceso a todos los estilos calculados del elemento.

Eventos DOM

Un **evento** ocurre cuando el usuario o el navegador realizan una acción: click, teclado, resize, carga de página, etc. Cuando ocurre, se lanza una función programable.

Tres formas de asociar eventos:

Forma	Característica
Atributos HTML	<code>onevento="funcion()"</code> ; código antiguo, mezcla JS con HTML.
Manejadores DOM	<code>element.onevento = funcion</code> ; solo permite una función por evento.

Forma	Característica
Listeners DOM	<code>element.addEventListener("event", funcion)</code> ; permite varias funciones y es la forma recomendada.

Flujo de eventos: si el evento ocurre en una etiqueta interior, puede propagarse hacia fuera (**bubbling**, `false`) o desde fuera hacia dentro (**capturing**, `true`).

El objeto evento permite consultar datos como `type` o `target`.

APIs JS

Una **API** permite a un programa interactuar con otros programas o scripts accediendo a sus datos.

Categoría	Ejemplos
APIs de navegador	DOM, Web Audio, WebRTC, Canvas, WebGL, Geolocation, Storage.
APIs de terceros	Facebook, Twitter, Google Maps, YouTube, etc.; no vienen incluidas por defecto.

APIs citadas:

- **Geolocation**: obtiene latitud y longitud si el usuario da permiso; se usa con `navigator.geolocation`.
- **localStorage** y **sessionStorage**: almacenamiento local en cliente, alternativo a cookies.
- **localStorage** no tiene fecha de expiración.
- **sessionStorage** dura la sesión y se pierde al cerrar la ventana.
- Datos de storage: se guardan con `setItem(nombreDato, valorDato)` o como atributo (`localStorage.nombreDato = valorDato`), y se recuperan con `getItem(nombreDato)` o `localStorage.nombreDato`.
- **history**: acceso al historial de páginas visitadas.
- **console**: depuración; `console.log()` registra valores o mensajes.

4.3 Sintaxis básica

Reglas generales, variables y tipos

JavaScript:

- distingue mayúsculas y minúsculas;
- usa comentarios como C: `//` y `/* ... */`;

- suele terminar instrucciones con `;`.

Variables:

Declaración	Ámbito de bloque	Redeclaración	Reasignación
<code>var nombre;</code>	No	Sí	Sí
<code>let nombre;</code>	Sí	No	Sí
<code>const nombre = valor;</code>	Sí	No	No

Tipos citados:

Tipo	Idea
<code>string</code>	Cadenas de texto.
<code>number</code>	Valores numéricos.
<code>boolean</code>	Booleanos.
<code>Object</code>	Objetos.
<code>null</code>	Nulo.
<code>undefined</code>	No definido.

`typeof(variable)` devuelve el tipo.

Conversión:

- implícita: coerción automática, frecuentemente hacia cadenas;
- explícita: `parseFloat()`, `parseInt()`, `toString()`, `String()`, `Number()`, etc.

Operadores, `Math` y control de flujo

Grupo	Operadores o elementos
Aritméticos	<code>-</code> , <code>+</code> , <code>*</code> , <code>/</code> , <code>++</code> , <code>--</code> , <code>%</code> .
Lógicos	AND, OR, NOT.
Relacionales	<code>==</code> , <code>!=</code> , <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , <code>===</code> , <code>!==</code> .
Bit a bit	<code>&</code> , <code>^</code>
Asignación	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>`</code>
Otros	<code>+</code> para concatenar strings, <code>.</code> , ternario <code>?:</code> , spread <code>...</code> .

`==` compara con conversión; `===` compara valor y tipo.

`Math` agrupa operaciones matemáticas:

- métodos: `log()`, `exp()`, `sqrt()`, `pow()`, `abs()`, `floor()`, `ceil()`, `round()`, `random()`, `sin()`, `cos()`, `tan()`, `max()`, `min()`, etc.;
- constantes: `E`, `LN2`, `LN10`, `LOG2E`, `LOG10E`, `PI`, `SQRT2`, `SQRT1_2`;
- el PDF cita `with(Math)` para no repetir el objeto, aunque hoy no suele recomendarse.

Estructuras de control:

Estructura	Uso
<code>if (...) { ... } else { ... }</code>	Condicional.
<code>switch (...) { case: ... break; }</code>	Selección múltiple.
<code>for(inicial; condicion; incremento)</code>	Bucle clásico.
<code>for (dato in objeto)</code>	Itera claves o índices.
<code>for (dato of iterable)</code>	Itera valores.
<code>while(condicion)</code>	Bucle con condición inicial.
<code>do { ... } while(condicion)</code>	Bucle que se ejecuta al menos una vez.

Funciones

Las funciones no especifican tipo de parámetros ni de retorno.

Forma	Idea
Declaración	<code>function nombre(parametros) { ... return ... }.</code>
Expresión anónima	<code>x = function(parametros) { ... }.</code>
Arrow function	<code>x = (parametros) => { ... };</code> si tiene un argumento se pueden omitir paréntesis.
Autoinvocada	<code>(function(parametros) { ... })(valores);</code> ; no necesita nombre ni <code>return</code> .
Callback	Una función pasada como argumento a otra.

Reglas importantes:

- se pueden establecer valores por defecto para parámetros;
- se permite número indefinido de parámetros con `...`;
- los primitivos se pasan por valor;
- los objetos, incluidos arrays, se pasan por referencia;
- las funciones son objetos y tienen `arguments`, `arguments.length` y `toString()`.

Objetos, arrays y cadenas

Objetos:

- tradicionalmente se instanciaban con `new`, invocando una función constructora;
- hoy se usa mucho el literal `{ atributo: valor, funcion() { ... } }`;
- las propiedades son públicas y se accede con `objeto.atributo` o `objeto["atributo"]`;
- los métodos se invocan con `objeto.funcion(parametros)`;
- los constructores pueden llamar a la clase padre con `super()`.

Arrays:

- son objetos;
- pueden crearse con `new`, pero se recomienda `let array = []`;
- tienen tamaño dinámico;
- son heterogéneos.

Método/propiedad	Uso
<code>length</code>	Tamaño del array.
<code>join()</code>	Une elementos en una cadena.
<code>reverse()</code>	Invierte orden.
<code>sort()</code>	Ordena.
<code>concat()</code>	Concatena arrays.
<code>slice()</code>	Extrae parte.
<code>Array.from()</code>	Crea array desde iterable.
<code>keys()</code>	Iterador con índices.
<code>find()</code>	Primer elemento que cumple una función.
<code>findIndex()</code>	Índice del primer elemento que cumple una función.
<code>forEach()</code>	Ejecuta una función por elemento.
<code>map()</code>	Crea un nuevo array transformado.
<code>filter()</code>	Crea array con elementos que devuelven <code>true</code> .
<code>every()</code>	<code>true</code> si todos cumplen.
<code>some()</code>	<code>true</code> si alguno cumple.
<code>reduce()</code>	Reduce de izquierda a derecha a un único valor.
<code>reduceRight()</code>	Reduce desde la derecha.

En `forEach`, `map`, `filter`, `every` y `some`, la función recibe `(value, index, array)`. En `reduce` y `reduceRight` se añade el acumulador o total.

Cadenas:

- son objetos;
- se recomienda `let cadena = "..."`, no `new String()`;
- métodos citados: `fromCharCode()`, `charCodeAt()`, `indexOf()`, `lastIndexOf()`, `substr()`, `slice()`, `substring()`, `toLowerCase()`, `toUpperCase()`, `includes()`, `startsWith()`, `endsWith()`.

Eventos básicos y `Date`

Eventos citados:

Evento	Significado
<code>click</code>	Pulsar ratón.
<code>mouseover</code> / <code>mouseout</code>	Entrar o salir con el ratón.
<code>load</code> / <code>unload</code>	Cargar o abandonar página.
<code>focus</code> / <code>blur</code>	Activación o pérdida de foco.
<code>keypress</code>	Presionar tecla.
<code>select</code>	Seleccionar.
<code>change</code>	Modificar.
<code>submit</code> / <code>reset</code>	Enviar o reiniciar formulario.
<code>error</code> / <code>abort</code>	Error o aborto de carga.

Eventos por elemento:

Elemento	Eventos citados
<code>window</code>	<code>load</code> , <code>unload</code> , <code>blur</code> , <code>focus</code> .
<code>link</code>	<code>click</code> , <code>mouseout</code> , <code>mouseover</code> .
<code>area</code>	<code>mouseOut</code> , <code>mouseOver</code> .
<code>image</code>	<code>abort</code> , <code>error</code> , <code>load</code> .
<code>form</code>	<code>reset</code> , <code>submit</code> .
<code>text</code> , <code>textarea</code> , <code>password</code>	<code>blur</code> , <code>change</code> , <code>focus</code> , <code>select</code> .
<code>button</code> , <code>reset</code> , <code>submit</code> , <code>radio</code> , <code>checkbox</code>	<code>click</code> .
<code>select</code>	<code>blur</code> , <code>change</code> , <code>focus</code> .
Controles de selección de archivos	<code>blur</code> , <code>focus</code> , <code>select</code> .

Fechas:

- se gestionan con `Date`;
- constructores citados: `new Date("mes, dia, ano, horas:minutos:segundos")` y `new Date(ano, mes, dia, horas, minutos, segundos, milisegundos)`;
- métodos: `getTime()`, `getDay()`, `getDate()`, `getMonth()`, `getYear()`, `getFullYear()`, `getHours()`, `getMinutes()`, `getSeconds()`, `getMilliseconds()`;
- métodos `set...` equivalentes;
- métodos UTC equivalentes;
- conversiones: `toUTCString()`, `toGMTString()`.

4.4 JavaScript ES6

En **2015**, ECMAScript tuvo una revisión importante llamada **ES6** o **ECMAScript 6**. Desde entonces las revisiones continúan anualmente, pero con menor intensidad.

ES6 introdujo:

- `let` y `const`;
- operador spread `...`;
- valores por defecto en funciones;
- número indefinido de parámetros;
- funciones resumidas con `=>`;
- bucles `for...of`;
- constructores sin escribir `function`;
- módulos;
- mapas y conjuntos;
- símbolos;
- nuevos métodos de `String`, `Array`, `Math`, `Number`;
- métodos globales `isFinite()` e `isNaN()`.

Módulos

Los **módulos** son ficheros que agrupan funciones o clases con un propósito común.

Reglas:

- suelen organizarse en carpetas separadas;
- se ha planteado la extensión `.mjs`, pero no está generalizada;
- se invocan a través de servidores;
- para usarlos en HTML se añade `type="module"` a `<script>`;
- se pueden importar nombres exportados, renombrarlos con `as`, importar un `default` o combinar `default` con nombres no `default`.

`class`, `Map`, `Set` y `Symbol`

ES6 introduce `class` como sintaxis moderna para clases. La herencia se hace con `extends` y `super()`. Los atributos siguen siendo públicos salvo que se usen mecanismos específicos modernos.

Mapas:

Método/propiedad	Uso
<code>new Map()</code>	Crea un mapa.
<code>set(clave, valor)</code>	Fija valor para una clave.
<code>get(clave)</code>	Recupera valor.
<code>clear()</code>	Borra todos los elementos.
<code>keys()</code>	Iterador de claves.
<code>values()</code>	Iterador de valores.

Los mapas son iterables en orden y sus claves pueden ser objetos.

Sets:

Método	Uso
<code>new Set()</code>	Crea un conjunto.
<code>add(valor)</code>	Añade valor.
<code>delete(valor)</code>	Elimina valor.
<code>clear()</code>	Borra todo.
<code>values()</code>	Iterador con valores.

Un `Set` almacena valores únicos.

`Symbol` es un tipo primitivo que representa un identificador único oculto, útil para evitar accesos accidentales. Se crea con `Symbol("nombre")`; se accede mediante `[]`, no mediante punto.

4.5 Expresiones regulares y validación

Durante el intercambio de datos con el servidor conviene enviar datos consistentes para no corromper la información almacenada. Las **expresiones regulares** permiten definir patrones de cadenas y usarlos para comprobar información.

Definición y clase `RegExp`

Formas de crear expresiones regulares:

Forma	Ejemplo
Literal	<code>/patron/opciones</code> .
Constructor	<code>new RegExp("patron", "opciones")</code> .

Opciones:

- `i`: no distingue mayúsculas/minúsculas;
- `g`: búsqueda global;
- `m`: búsqueda multilínea.

Clase `RegExp`:

Tipo	Elemento	Uso
Método	<code>exec()</code>	Comprueba coincidencias y retorna la primera.
Método	<code>test()</code>	Devuelve <code>true</code> o <code>false</code> .
Método	<code>toString()</code>	Devuelve la expresión como cadena.
Propiedad	<code>global</code>	Indica si <code>g</code> está activado.
Propiedad	<code>ignoreCase</code>	Indica si <code>i</code> está activado.
Propiedad	<code>multiline</code>	Indica si <code>m</code> está activado.
Propiedad	<code>lastIndex</code>	Posición de inicio de la siguiente coincidencia.
Propiedad	<code>source</code>	Patrón como cadena.

Métodos de `String` con regex:

Método	Uso
<code>match()</code>	Busca coincidencias y devuelve array.
<code>matchAll()</code>	Devuelve iterador con coincidencias.
<code>search()</code>	Devuelve índice de la primera coincidencia.
<code>replace()</code>	Reemplaza texto usando regex.
<code>split()</code>	Divide la cadena usando regex como separador.

Patrones y precedencia

Patrón	Significado	
<code>*</code>	Cero o más veces.	
<code>+</code>	Una o más veces.	
<code>?</code>	Cero o una vez.	
<code>\</code>	Carácter de escape.	
<code>[conjunto]</code>	Cualquier carácter del conjunto.	
<code>\b</code>	Final de palabra.	
<code>\B</code>	No final de palabra.	
<code>^</code>	Comienzo de cadena o línea.	
<code>\$</code>	Final de cadena o línea.	
<code>,</code>	<code>,</code>	Alternancia.
<code>(patron)</code>	Agrupación o captura.	
<code>(?:...)</code>	Subpatrón sin captura.	
<code>(?=...)</code>	Coincidencia si va seguida de otra expresión.	
<code>(?!...)</code>	Coincidencia si no va seguida de otra expresión.	
<code>{n}</code>	Exactamente <code>n</code> veces.	
<code>{n,}</code>	Como mínimo <code>n</code> veces.	
<code>{n,m}</code>	Entre <code>n</code> y <code>m</code> veces.	
<code>\d</code>	Equivale a <code>[0-9]</code> .	
<code>\w</code>	Equivale a letras, números y <code>_</code> .	
<code>\W</code>	Negación de <code>\w</code> .	
<code>\n, \r, \t, \f, \v, \s</code>	Abreviaturas de caracteres especiales o espacios.	

Precedencia:

1. `\`
2. `()`, `[]`
3. `*`, `+`, `{n}`, `{n,}`, `{n,m}`
4. `^`, `$`
5. `|`

Validación

Validación con JavaScript:

- interceptar envío de formulario;
- usar expresiones regulares;
- cancelar comportamiento con `preventDefault()`;
- avisar con `alert()`;
- devolver foco con `focus()` o seleccionar texto con `select()`.

Validación HTML5:

Atributo/tipo	Uso
<code>type="email"</code>	Campo de email con validación básica.
<code>type="url"</code>	Validación similar para URL.
<code>type="number"</code>	Validación similar para número.
<code>pattern</code>	Permite introducir una expresión regular.
<code>required</code>	Campo obligatorio; es booleano.

4.6 jQuery

jQuery es una librería de funciones escrita en JavaScript, libre y de código abierto, creada por **John Resig** y presentada en enero de **2006** en BarCamp NYC.

Permite:

- simplificar interacción con documentos HTML;
- manipular el DOM;
- manejar eventos;
- crear animaciones;
- trabajar con AJAX.

Ventajas frente al DOM nativo:

- selecciones HTML más simples;
- varios métodos sobre la misma selección;
- robustez frente a diferencias entre navegadores;
- manejo de eventos más sencillo;
- métodos adicionales para animaciones;
- gestión de inconsistencias entre navegadores.

Puede integrarse descargando `jquery.js` o enlazándolo desde un CDN.

Sintaxis y selectores

Sintaxis básica:

```
$(selector).action()
```

Parte	Significado
<code>\$</code>	Abreviatura de <code>jQuery()</code> .
<code>selector</code>	Selector con sintaxis CSS.
<code>action()</code>	Método aplicado al objeto jQuery.
<code>length</code>	Número de etiquetas seleccionadas.

`ready()` ejecuta una función cuando el DOM está preparado: `$(document).ready(function(){ ... })`; la forma abreviada es `$(function(){ ... })`.

Selectores específicos:

Selector	Uso
<code>\$(this)</code>	Etiqueta actual.
<code>\$("p:first")</code> / <code>\$("p:last")</code>	Primera o última etiqueta <code>p</code> .
<code>\$("ul li:first-child")</code>	Primer <code>li</code> de cada <code>ul</code> .
<code>\$(":button")</code>	<code>button</code> e <code>input type="button"</code> .
<code>\$("tr:even")</code> / <code>\$("tr:odd")</code>	Filas pares o impares.

Métodos jQuery

Grupo	Métodos
Contenido	<code>html()</code> , <code>text()</code> , <code>replaceWith()</code> , <code>remove()</code> .
Nodos	<code>before()</code> , <code>after()</code> , <code>prepend()</code> , <code>append()</code> , <code>clone()</code> , <code>unwrap()</code> , <code>detach()</code> , <code>empty()</code> , <code>add()</code> .
Atributos y estilos	<code>attr()</code> , <code>removeAttr()</code> , <code>addClass()</code> , <code>removeClass()</code> , <code>css()</code> .
Formularios	<code>val()</code> , <code>isNumeric()</code> .
Recorrido	<code>find()</code> , <code>closest()</code> , <code>parent()</code> , <code>parents()</code> , <code>children()</code> , <code>siblings()</code> , <code>next()</code> , <code>nextAll()</code> , <code>prev()</code> , <code>prevAll()</code> .
Lazos	<code>each()</code> ; los ítems se acceden con <code>this</code> .

Grupo	Métodos
Testeo y filtrado	<code>filter()</code> , <code>not()</code> , <code>has()</code> , <code>is()</code> , <code>\$.contains(padre, hijo)</code> .
Orden	<code>eq()</code> , <code>lt()</code> , <code>gt()</code> .
Dimensión	<code>height()</code> , <code>width()</code> , <code>innerHeight()</code> , <code>innerWidth()</code> , <code>outerHeight()</code> , <code>outerWidth()</code> , <code>\$(document).height()</code> , <code>\$(document).width()</code> , <code>\$(window).height()</code> , <code>\$(window).width()</code> .
Posición	<code>offset()</code> , <code>position()</code> , <code>scrollLeft()</code> , <code>scrollTop()</code> .
Animación	<code>show()</code> , <code>hide()</code> , <code>toggle()</code> , <code>fadeIn()</code> , <code>fadeOut()</code> , <code>fadeTo()</code> , <code>fadeToggle()</code> , <code>slideDown()</code> , <code>slideUp()</code> , <code>slideToggle()</code> , <code>delay()</code> , <code>stop()</code> , <code>animate()</code> .

Notas:

- `html()` interpreta HTML; `text()` no lo interpreta.
- `replaceWith()` reemplaza el nodo entero.
- `detach()` es como `remove()`, pero conserva una copia en memoria.
- `height()` y `width()` no incluyen margen, borde ni padding.
- `innerHeight()` e `innerWidth()` no incluyen margen ni borde.
- `outerHeight()` y `outerWidth()` no incluyen margen.
- `offset()` se refiere a la esquina superior izquierda del documento; `position()` al ancestro.
- todos los elementos retornados por una selección tienen índice.

Eventos jQuery

Dos formas:

- `$(selector).evento(function(e){ ... });`
- `$(selector).on("evento", function(e){ ... });`

`on()` es la forma moderna y recibe el evento y la función manejadora.

Objeto evento:

Propiedad/método	Uso
<code>type</code>	Tipo de evento.
<code>which</code>	Botón o tecla que lo desencadena.
<code>target.nodeName</code>	Nombre del elemento seleccionado.
<code>pageX</code>	Posición del ratón desde el lado izquierdo del documento.
<code>pageY</code>	Posición del ratón desde la parte superior del documento.
<code>preventDefault()</code>	Evita la acción por defecto.
<code>stopPropagation()</code>	Detiene la propagación hacia ancestros.

Tipos de eventos:

Grupo	Eventos
UI	<code>focus</code> , <code>blur</code> , <code>change</code> .
Teclado	<code>input</code> , <code>keydown</code> , <code>keyup</code> , <code>keypress</code> .
Ratón	<code>click</code> , <code>dblclick</code> , <code>mouseup</code> , <code>mousedown</code> , <code>mouseover</code> , <code>mousemove</code> , <code>mouseout</code> , <code>hover</code> .
Formulario	<code>submit</code> , <code>select</code> , <code>change</code> .
Documento	<code>ready</code> , <code>load</code> , <code>unload</code> .
Navegador	<code>error</code> , <code>resize</code> , <code>scroll</code> .

4.7 Ficheros de datos: XML y JSON

XML

XML (eXtensible Markup Language) es un lenguaje de marcas que describe información estructurada para facilitar el intercambio entre máquinas. Existen herramientas de fabricantes como Microsoft o Apache que facilitan su uso.

Archivos de propiedades	Archivos XML
Jerarquía plana.	Estructuras jerárquicas con etiquetas.
No admiten repetición de valores.	Admiten estructuras repetidas y anidadas.

XML frente a HTML:

HTML	XML
Etiquetas predefinidas por estándar.	Etiquetas no predefinidas.
No distingue mayúsculas/minúsculas.	Sí distingue mayúsculas/minúsculas.
Existen atributos sin valor.	Todo atributo debe tener valor.
Más flexible con cierres.	Toda etiqueta debe cerrarse; elementos vacíos terminan en <code></></code> .
Valores de atributos más flexibles.	Comillas simples o dobles, pero deben abrirse y cerrarse igual.

Estructura XML:

1. Encabezado opcional pero recomendado: `<?xml version="1.0" encoding="UTF-8"?>`.
2. Definición de tipo de documento opcional, por ejemplo un `DOCTYPE`.
3. Cuerpo con elementos hijos, texto o ambas cosas.

Consejos:

- evitar contenido mixto, es decir, texto y elementos al mismo nivel;
- usar atributos solo para modificar la interpretación de un valor, no para especificar valores.

XML DOM

El **XML DOM** define un estándar para acceder y manipular XML de manera similar al HTML DOM.

Objeto/método/propiedad	Uso
<code>DOMParser</code>	Parsea XML almacenado en una cadena y lo convierte en documento DOM.
<code>XMLHttpRequest.responseXML</code>	Devuelve un <code>XMLDocument</code> con el XML de la petición.
<code>parseFromString(texto, tipoMime)</code>	Método de <code>DOMParser</code> que devuelve un <code>XMLDocument</code> .
<code>getElementsByTagName(nombre)</code>	Devuelve <code>NodeList</code> con etiquetas de ese nombre.
<code>appendChild(nodo)</code>	Inserta nodo hijo.
<code>removeChild(nodo)</code>	Elimina nodo hijo.

Objeto/método/propiedad	Uso
<code>nodeName</code>	Nombre del nodo.
<code>nodeValue</code>	Valor del nodo.
<code>nodeType</code>	Tipo del nodo.
<code>parentNode</code>	Padre del nodo.
<code>childNodes</code>	Hijos del nodo.
<code>attributes</code>	<code>NamedNodeMap</code> con atributos.

JSON

JSON (JavaScript Object Notation) es un formato de almacenamiento e intercambio de datos creado inicialmente en 1999 en el ámbito de JS y extendido a muchos lenguajes.

Características:

- sencillo para máquinas y humanos;
- en JS se maneja como objeto;
- puede convertirse a cadena para intercambio entre servidores;
- `JSON.stringify(obj)` transforma objetos JS en cadenas JSON;
- `JSON.parse(cadena)` transforma cadenas JSON en objetos JS.

Formato:

Estructura	En JS
Colección de pares nombre/valor	Objetos: <code>{ cadena: valor, ... }</code> .
Lista ordenada de valores	Arrays: <code>[valor, valor, ...]</code> .

Valores admitidos: cadenas, números, objetos, arrays, booleanos y `null`.

XML vs JSON

Similitudes:

- ambos son autodescriptivos;
- ambos son jerárquicos;
- ambos son parseables;
- ambos son tratables mediante `XMLHttpRequest`.

Diferencias:

- XML necesita parser XML; JSON usa funciones estándar de JS;
- JSON no usa etiquetas;
- JSON es más rápido en lectura y escritura;
- JSON puede usar arrays.

4.8 AJAX, promesas y Fetch

MPA, SPA y procesamiento asíncrono

Modelo	Idea
MPA	Multi-Page Application: cada página es una petición al servidor que devuelve un HTML completo.
SPA	Single-Page Application: una única página HTML sobrescribe contenido dinámicamente sin cargar una nueva página.

Procesamiento:

- **síncrono**: al encontrar `<script>`, el navegador lo procesa y detiene la carga del resto de la página; si el script espera datos externos, la carga se bloquea;
- **asíncrono**: AJAX carga datos en segundo plano y permite continuar con el resto del documento.

AJAX (Asynchronous JavaScript And XML) es una técnica para acceder al servidor desde la página web. Nace en febrero de **2005** de la mano de **Jesse James Garrett**.

Permite:

- actualizar una página sin recargarla;
- pedir datos al servidor después de cargar la página;
- recibir datos después de cargar la página;
- enviar datos en background;
- desarrollar SPA.

AJAX combina:

- JS y HTML DOM para mostrar datos;
- un objeto `XMLHttpRequest` construido en el navegador.

El **motor AJAX** es el objeto o función JS invocado cuando se realiza una petición al servidor.

`XMLHttpRequest`

`XMLHttpRequest` fue diseñado por Microsoft y hoy es estándar W3C. Sirve para insertar datos en una página sin recargarla.

Métodos:

Método	Uso
<code>abort()</code>	Cancela la solicitud.
<code>getAllResponseHeaders()</code>	Devuelve cabeceras.
<code>getResponseHeader(nombre)</code>	Devuelve cabecera concreta.
<code>open(method, url, async, user, psw)</code>	Especifica solicitud: método <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code> ; URL; modo asíncrono; usuario y contraseña opcionales.
<code>send()</code>	Envía solicitud GET.
<code>send(string)</code>	Envía solicitud POST.
<code>setRequestHeader(header, value)</code>	Añade par etiqueta/valor a la cabecera.

Atributos:

Atributo	Uso
<code>onreadystatechange</code>	Callback cuando cambia <code>readyState</code> .
<code>onload</code>	Callback cuando se recibe la petición; navegadores modernos lo soportan y suele usarse.
<code>readyState</code>	Estado de la petición.
<code>responseText</code>	Respuesta en texto.
<code>responseXML</code>	Respuesta en XML.
<code>status</code>	Código de estado: <code>200</code> OK, <code>403</code> prohibido, <code>404</code> no encontrado, etc.
<code>statusText</code>	Texto del estado.

`readyState`:

Valor	Estado
<code>0</code>	Petición no inicializada.
<code>1</code>	Conexión con servidor establecida.
<code>2</code>	Petición recibida.
<code>3</code>	Procesando petición.
<code>4</code>	Petición finalizada y respuesta lista.

Funcionamiento:

1. crear objeto: `let miHttp = new XMLHttpRequest();`
2. definir callback con `onreadystatechange` u `onload`;

3. abrir con `open("GET", "miFile");`
4. enviar con `send()`.

Promesas y AJAX con jQuery

Una **promesa** representa la terminación correcta o el fracaso de una operación asíncrona.

Forma general:

```
new Promise(function(siExito, siFracaso) { ... siExito(valor); siFracaso(valor); })
```

`then(siExito, siFracaso)` define callbacks de éxito o fracaso.

AJAX con jQuery usa `load()`:

```
.load(url, [data, function(response, status, xhr)])
```

Parámetro	Uso
<code>url</code>	URL de datos a cargar.
<code>data</code>	Datos enviados junto a la petición.
<code>response</code>	Datos resultantes.
<code>status</code>	Estado de la petición.
<code>xhr</code>	Objeto <code>XMLHttpRequest</code> .

Fetch

Desventajas de `XMLHttpRequest`:

- diferencias entre fabricantes o versiones de navegador;
- uso de callbacks, que puede desordenar el código.

La solución tradicional era usar herramientas de terceros, como jQuery. **Fetch** es una interfaz estándar para AJAX basada en promesas.

`window.fetch(url)` o `fetch(url)` trae al cliente el contenido de la URL y devuelve una promesa que resuelve en un objeto `Response`, incluso si el servidor devuelve un código de error HTTP.

Objeto `Response`:

Método/propiedad	Uso
<code>.text()</code>	Promesa con representación en texto.
<code>.json()</code>	Promesa con representación JSON.
<code>.formData()</code>	Promesa con pares clave/valor.
<code>.bytes()</code>	Promesa con bytes.
<code>.status</code>	Código de estado.
<code>.ok</code>	Booleano de éxito; normalmente <code>200</code> a <code>299</code> .
<code>.headers</code>	Cabeceras de la respuesta.
<code>.url</code>	URL de la respuesta.

- `fetch()` no rechaza la promesa por un `404` o un `500`; conviene comprobar `response.ok`;
- para texto se encadena `response.text()`;
- para JSON se usa `response.json()`;
- para XML se puede recibir texto y parsearlo con `DOMParser`.

5. Servlets

5.1 Aplicaciones web en Java

Concepto	Definición
Sitio web	Conjunto de páginas web almacenadas en un servidor y relacionadas por contenido. Usa principalmente tecnologías front-end: HTML, CSS y JS.
Aplicación web	Conjunto de páginas generadas en respuesta a peticiones del usuario, normalmente mediante formularios. Requiere front-end y back-end.
Página HTML estática	Página almacenada en el servidor cuyo contenido no depende de la llamada.
Página HTML dinámica	Página generada en el servidor cuyo contenido puede depender de la llamada.

Para desarrollar aplicaciones web en Java se usa **Java EE / Jakarta EE**, una especificación construida sobre **Java SE**.

Elementos necesarios:

Elemento	Incluye o aporta
JDK	Compilador de Java, clases core y JRE.
APIs Java EE/Jakarta EE	Servlets, JSP, JSF, JPA, EJB, etc.

Definiciones útiles:

Concepto	Papel habitual
Servlet	Clase Java gestionada por el contenedor web; atiende HTTP y suele actuar como controlador.
JSP	Java Server Pages: página HTML con etiquetas y fragmentos Java que el servidor traduce a un servlet; suele actuar como vista.
JavaBean	Clase reutilizable con constructor vacío, atributos privados y <code>get/set</code> ; encapsula datos o lógica sencilla del modelo.

Enfoques habituales:

Enfoque	Nivel	Control sobre HTML/CSS/JS
Servlets + JSP	Bajo nivel; hace poco trabajo por el programador.	Alto control.
JSF	Alto nivel; delega mucho trabajo.	Menor control.
Frameworks como Spring o Struts	Alto nivel; delegan trabajo.	Alto control.

MVC

MVC (Modelo-Vista-Controlador) es un patrón introducido por **Trygve Reenskaug en 1979**.

Parte	Responsabilidad
Modelo	Datos, lógica de negocio, acceso a datos, reglas de negocio y funciones de base de datos.
Vista	Interfaz de usuario y presentación de datos.
Controlador	Intermediario: recibe entradas, las procesa, actualiza modelo y decide la vista.

Flujo MVC general:

1. El usuario interactúa con la vista.
2. El controlador procesa la acción.
3. El modelo actualiza los datos.
4. La vista se actualiza y muestra la información.

En Java web clásico:

- **Servlet** = controlador.
- **JSP/HTML** = vista.
- **JavaBeans, DAOs y clases de negocio** = modelo.

Flujo web con Java:

1. El usuario solicita una URL mediante un objeto `request`.
2. El servlet controlador atiende la solicitud.
3. El controlador obtiene datos del modelo.
4. Añade datos a `request` o prepara `response`.
5. La vista muestra los datos mediante HTML/JSP.

5.2 Qué es un servlet

Un **contenedor web** es la interfaz entre un componente y el sistema inferior que lo soporta. Gestiona ciclo de vida, peticiones, objetos `request/response`, mapeo de URLs y ejecución dentro del servidor. Ejemplos: **Tomcat** o **Jetty**.

Un **servlet** es un programa Java que se ejecuta en el contenedor web de un servidor de aplicaciones y es invocado por un cliente mediante HTTP.

Características:

- independiente de plataforma, al estar escrito en Java;
- se carga en memoria la primera vez y en peticiones posteriores se usan hilos;
- consume menos recursos que CGI;
- está precompilado;
- se ejecuta dentro de la JVM, con excepciones y seguridad Java;
- es portable y seguro;
- es invisible para el cliente: el navegador no necesita Java.

Idea práctica importante: normalmente existe una instancia del servlet y varias peticiones concurrentes la usan con hilos; por eso no se debe guardar estado mutable de usuario en atributos de instancia.

API Servlet

Desde el punto de vista del código, un servlet pertenece a la API Java Servlet e implementa la interfaz `Servlet`.

Paquete clásico	Contenido
<code>javax.servlet</code>	Clases e interfaces genéricas, independientes del protocolo.
<code>javax.servlet.http</code>	Clases e interfaces específicas para HTTP.

En versiones actuales se usa `jakarta.servlet` y `jakarta.servlet.http`.

Jerarquía habitual:

Elemento	Papel
<code>Servlet</code>	Interfaz base.
<code>GenericServlet</code>	Implementa servicio genérico, no limitado a HTTP.
<code>HttpServlet</code>	Especialización para HTTP; hereda de <code>GenericServlet</code> .
Clase propia	Servlet escrito por el desarrollador.

Los métodos del ciclo de vida y de petición los invoca el **contenedor**, no el programador.

5.3 Ciclo de vida y procesamiento de peticiones

Ciclo de vida

Método	Cuándo se ejecuta	Uso
<code>init(ServletConfig)</code>	Una vez, cuando el servidor arranca o inicializa el servlet.	Inicializar recursos.
<code>service(request, response)</code>	En cada petición.	Crear la respuesta; en HTTP despacha hacia <code>doGet()</code> , <code>doPost()</code> , etc.
<code>destroy()</code>	Una vez, antes de apagar o descargar el servlet.	Cerrar conexiones, liberar memoria.

Cómo maneja el contenedor una petición

1. El usuario hace clic en una URL asociada a un servlet.
2. El contenedor comprueba si la petición es para un servlet.
3. Crea objetos `HttpServletRequest` y `HttpServletResponse`.
4. Encuentra el servlet correcto según la URL, usando `web.xml` o anotaciones.
5. Crea un hilo para la petición y le pasa `request` y `response`.
6. Ejecuta `doGet()` o `doPost()`.
7. El hilo termina.
8. El contenedor convierte `response` en respuesta HTTP y la devuelve al cliente.
9. El contenedor elimina los objetos de petición y respuesta.

Métodos principales

`GenericServlet`:

Método	Uso
<code>getServletConfig()</code>	Configuración de arranque pasada a <code>init()</code> .
<code>getServletContext()</code>	Contexto de la aplicación.
<code>getInitParameter(nombre)</code>	Valor de un parámetro de inicialización.
<code>getInitParameterNames()</code>	Nombres de parámetros disponibles.
<code>getServletInfo()</code>	Información general: autor, versión, copyright, etc.
<code>log(mensaje)</code>	Escribe en el log del servidor.

Método	Uso
<code>log(mensaje, excepcion)</code>	Escribe mensaje y traza de error.

`HttpServlet`:

Método	Uso
<code>doGet(request, response)</code>	Atiende GET.
<code>doPost(request, response)</code>	Atiende POST.
<code>doHead(request, response)</code>	Como GET, pero solo devuelve cabeceras.
<code>doOptions(request, response)</code>	Indica métodos permitidos en la URL.
<code>doPut(request, response)</code>	Atiende PUT.
<code>doTrace(request, response)</code>	Devuelve la misma petición recibida.
<code>doDelete(request, response)</code>	Atiende DELETE.
<code>getLastModified(request)</code>	Fecha en milisegundos del último cambio del recurso.

5.4 `HttpServletRequest` y `HttpServletResponse`

`HttpServletRequest` encapsula datos enviados por el cliente. `HttpServletResponse` encapsula datos enviados por el servidor al cliente.

`HttpServletRequest`

Método	Uso
<code>getAttribute(nombre)</code>	Valor de un atributo guardado en la petición.
<code>getParameter(nombre)</code>	Valor enviado por URL o formulario.
<code>getParameterNames()</code>	Nombres de parámetros enviados.
<code>getMethod()</code>	Método HTTP usado.
<code>getContentTypeLength()</code>	Tamaño exacto en bytes del cuerpo.
<code>getInputStream()</code>	Canal de entrada byte a byte.
<code>getLocalPort()</code>	Puerto del servidor por el que entró la petición.
<code>getContextPath()</code>	Parte de la URL que identifica la aplicación.
<code>getHeader(nombre)</code>	Cabecera enviada por el navegador.
<code>getQueryString()</code>	Parte de la URL posterior a <code>?</code> .
<code>getSession()</code>	Sesión actual, o creación de una nueva.
<code>getCookies()</code>	Cookies enviadas por el navegador.

HttpServletResponse

Método	Uso
<code>getOutputStream()</code>	Salida binaria.
<code>getWriter()</code>	<code>PrintWriter</code> para texto o HTML.
<code>sendError(codigo)</code>	Interrumpe y envía error HTTP estándar.
<code>setStatus(codigo)</code>	Establece código HTTP sin interrumpir la generación.
<code>getBufferSize()</code>	Tamaño del buffer.
<code>setContentType(tipo)</code>	Tipo de contenido enviado.
<code>addCookie(cookie)</code>	Ordena guardar una cookie.
<code>addHeader(nombre, valor)</code>	Añade cabecera técnica.
<code>encodeRedirectURL(url)</code>	Incrusta ID de sesión en URL si es necesario.

`getWriter()` se usa para texto; `getOutputStream()` para datos binarios.

5.5 Estructura de aplicación y configuración

Estructura clásica

```
miAplicacion/  
├─ index.html  
├─ css/  
├─ img/  
└─ WEB-INF/  
    ├─ web.xml  
    ├─ classes/  
    └─ lib/
```

Ideas:

- `WEB-INF` no es accesible directamente por URL.
- `classes/` contiene clases compiladas.
- `lib/` contiene `.jar`.
- `web.xml` es el descriptor de despliegue.

Estructura Maven

```
proyecto/  
├─ pom.xml  
└─ src/
```

```

├─ main/
│   ├── java/
│   ├── resources/
│   └── webapp/
│       └── WEB-INF/
└─ test/

```

Maven automatiza dependencias, compilación y empaquetado `.war`. Esta información no aparece tan desarrollada en el PDF, pero es útil para proyectos actuales.

`web.xml` y anotaciones

Tradicionalmente se configuraba el servlet en `web.xml` con `<servlet>`, `<servlet-name>`, `<servlet-class>`, `<servlet-mapping>` y `<url-pattern>`.

Desde **Servlet 3.0 (2009)** se puede usar `@WebServlet`, moviendo parte de la configuración al código Java:

<code>web.xml</code>	<code>@WebServlet</code>	Descripción
<code><servlet-name></code>	<code>name</code>	Nombre del servlet.
<code><url-pattern></code>	<code>urlPatterns</code>	Array de URLs que activan el servlet.
<code><url-pattern></code>	<code>value</code>	Igual que <code>urlPatterns</code> , pero para un solo valor.
<code><init-param></code>	<code>initParams</code>	Parámetros de inicialización con <code>@WebInitParam(name="...", value="...")</code> .

`@Override` es otro ejemplo de anotación Java, usada para evitar errores al sobrescribir métodos.

Conclusión:

- las anotaciones reducen XML;
- en aplicaciones complejas puede seguir usándose `web.xml`;
- las anotaciones incrementan dependencia de librerías externas si no se gestionan bien;
- los gestores de proyectos automatizan descarga de dependencias, versiones y empaquetado `.war`;
- frameworks modernos suelen apoyarse en gestores de proyectos.

5.6 Parámetros, contexto y atributos

Un **parámetro** es un dato `String` de configuración. Evita recompilar código para cambiar valores.

Parámetros de inicialización

Aspecto	Idea
Dónde se definen	<code><init-param></code> dentro de <code><servlet></code> .
Alcance	Un servlet.
Objeto de acceso	<code>ServletConfig</code> .
Inicialización	<code>ServletConfig config = getServletConfig();</code>
Métodos	<code>getInitParameter(nombre)</code> , <code>getInitParameterNames()</code> , <code>getServletName()</code> , <code>getServletContext()</code> .

Parámetros de contexto

Aspecto	Idea
Dónde se definen	<code><context-param></code> dentro de <code><web-app></code> .
Alcance	Toda la aplicación.
Objeto de acceso	<code>ServletContext</code> .
Inicialización	<code>getServletContext()</code> o <code>getServletConfig().getServletContext()</code> .
Métodos	<code>getInitParameter(nombre)</code> , <code>getInitParameterNames()</code> , <code>setAttribute()</code> , <code>getAttribute()</code> , <code>removeAttribute()</code> .

`ServletConfig` vs `ServletContext`

Aspecto	<code>ServletConfig</code>	<code>ServletContext</code>
Alcance	Un servlet.	Toda la aplicación.
Instancias	Una por servlet.	Una por aplicación.
Inicializado por	Contenedor al crear el servlet.	Contenedor al desplegar la aplicación.
Parámetros	<code><init-param></code> dentro de <code><servlet></code> .	<code><context-param></code> dentro de <code><web-app></code> .
Acceso	<code>getServletConfig()</code> .	<code>getServletContext()</code> .

Flujo:

1. Despliegue de la aplicación.
2. Creación de `ServletContext`.
3. Carga de `<context-param>`.
4. Creación de cada servlet.

5. Creación de su `ServletConfig`.
6. Llamada a `servlet.init(config)`.

Atributos

Un **atributo** es un objeto con nombre, valor y periodo de validez que se almacena en memoria del servidor para compartir información entre componentes.

Ámbito	Acceso	Significado
Aplicación	<code>getServletContext().getAttribute()</code>	Afecta a todos los servlets de la web.
Petición	<code>request.getAttribute()</code>	Afecta a la petición actual.
Sesión	<code>request.getSession().getAttribute()</code>	Afecta al usuario de esa sesión.

Métodos comunes:

- `getAttribute(nombre)`;
- `setAttribute(nombre, objeto)`;
- `removeAttribute(nombre)`;
- `getAttributeNames()`.

Resumen: **parámetros** = configuración, **atributos** = datos compartidos en ejecución.

5.7 Construcción de respuesta y redirección

Cuando un servlet toma el control puede:

1. implementar directamente la respuesta con `PrintWriter`;
2. pasar el control a otro recurso con `RequestDispatcher`;
3. redirigir al navegador con `HttpServletResponse`.

`PrintWriter`

`PrintWriter` devuelve cadenas de texto al navegador:

- inicialización: `PrintWriter wr = response.getWriter()`;
- `print(string)`;
- `println(string)`.

Mezcla lógica y presentación, por eso en aplicaciones medias suele preferirse JSP como vista.

RequestDispatcher

Permite enviar la solicitud a otro recurso del mismo servidor: HTML, servlet o JSP.

- inicialización: `RequestDispatcher rd = request.getRequestDispatcher(resource);`
- `forward(request, response)`: reenvía la petición y pasa el control;
- `include(request, response)`: incluye contenido en la respuesta y no pierde el control;
- `request` y `response` pueden ser modificados por ambos recursos.

sendRedirect()

`response.sendRedirect(resource)` redirige la respuesta a otro recurso. Acepta URLs relativas o absolutas y genera una nueva petición HTTP, con nuevos objetos `request` y `response`.

<code>forward()</code>	<code>sendRedirect()</code>
Se ejecuta en el servidor.	Se ejecuta desde el cliente.
Conserva los mismos <code>request</code> y <code>response</code> .	Genera nueva solicitud.
Solo puede usarse con recursos del servidor.	Puede apuntar dentro o fuera del servidor.
La URL del navegador no cambia.	La URL del navegador cambia.

5.8 Sesiones y seguimiento de estado

HTTP es un protocolo **sin estado**: cada petición se trata de forma independiente. El servidor no sabe por sí mismo si dos peticiones pertenecen al mismo cliente ni si están relacionadas.

Una **sesión** permite identificar y guardar información individual de cada cliente. Empieza con una petición y puede terminar por cierre, inactividad o métodos de fin de sesión.

Mecanismos de seguimiento de sesión

Mecanismo	Ventajas	Inconvenientes	Situación actual
Campos ocultos	No dependen de cookies; funcionan con formularios.	Manuales, difíciles de mantener, requieren formularios, solo texto.	Obsoleto.
Reescritura de URL	No depende de cookies, no necesita formularios,	Solo enlaces, URLs largas, problemas de seguridad, SEO y	Respaldo.

Mecanismo	Ventajas	Inconvenientes	Situación actual
	<code>encodeURIComponent()</code> puede automatizar.	caché.	
Cookies	No ensucian URL, más estándar y transparente.	Pueden rechazarse, privacidad, tamaño limitado, mantenimiento.	Muy habitual.
<code>HttpSession</code>	Fácil, integrada, gestiona cookies y reescritura.	Consumo de memoria y problemas de escalabilidad si se gestiona mal.	Recomendada.

Campos ocultos

Consiste en almacenar información en un campo `hidden` de un formulario y recuperarla con `request.getParameter()`.

```
<input type="hidden" name="id" value="123">
```

Reescritura de URL

Añade un token o identificador a la URL del siguiente recurso:

```
url?name1=value1&name2=value2
```

Se recupera con `getParameter()`. Puede automatizarse con `response.encodeURL()`, y `HttpSession` la soporta como respaldo.

Cookies

Una **cookie** es un fragmento de información almacenado en el cliente a petición del servidor. El navegador la devuelve en solicitudes posteriores.

Métodos:

Método	Uso
<code>new Cookie(nombre, valor)</code>	Crea cookie.
<code>response.addCookie(cookie)</code>	Envía cookie al cliente.
<code>getName()</code> / <code>setName()</code>	Nombre.
<code>getValue()</code> / <code>setValue()</code>	Valor.
<code>getMaxAge()</code> / <code>setMaxAge(segundos)</code>	Tiempo máximo; negativo implica cookie de sesión.
<code>getDomain()</code> / <code>setDomain()</code>	Dominio válido.

Método	Uso
<code>getSecure()</code>	<code>true</code> si solo viaja por HTTPS.
<code>setPath(ruta)</code>	Restringe uso a una ruta.

HttpSession

`HttpSession` deja que el contenedor cree un identificador de sesión, normalmente mediante la cookie predefinida `JSESSIONID`, y mantenga el estado automáticamente.

Inicialización:

- `request.getSession()` o `request.getSession(true)`: devuelve sesión actual o crea una nueva;
- `request.getSession(false)`: devuelve sesión actual, pero no crea una si no existe.

Métodos:

Método	Uso
<code>setAttribute(nombre, valor)</code>	Guarda dato en sesión.
<code>getAttribute(nombre)</code>	Recupera dato.
<code>removeAttribute(nombre)</code>	Elimina dato.
<code>getId()</code>	Identificador único.
<code>isNew()</code>	<code>true</code> si acaba de crearse y el navegador aún no la conoce.
<code>invalidate()</code>	Destruye la sesión y sus atributos.
<code>getServletContext()</code>	Contexto global de la aplicación.
<code>getCreationTime()</code>	Fecha/hora de creación.
<code>getLastAccessedTime()</code>	Último acceso del cliente.
<code>getMaxInactiveInterval()</code> / <code>setMaxInactiveInterval(segundos)</code>	Tiempo máximo de inactividad.

Finalización:

- en `web.xml` con `<session-config><session-timeout>X</session-timeout></session-config>`;
- con `session.setMaxInactiveInterval(segundos)`;
- con `session.invalidate()`.

5.9 Evolución de Servlets y migración a Jakarta

Versión	Año	Novedades
Servlet 3.0	2009	Anotaciones, asíncrono, pluggability, menos XML.
Servlet 3.1	2013	Soporte WebSocket, mejoras asíncronas, NonBlocking IO.
Servlet 4.0	2017	HTTP/2 y Server Push.
Servlet 5.0	2020	Migración a Jakarta EE, mejoras de seguridad, compatibilidad con Jakarta EE 9, limpieza de APIs antiguas.
Servlet 6.0	2022	Alta compatibilidad con HTTP/2, mejoras en headers, optimizaciones cloud-native.

De `javax` a `jakarta`

En **2017**, Oracle donó Java EE a la comunidad mediante la **Fundación Eclipse**, pero conservó la marca Java y prohibió modificar el paquete `javax`.

Consecuencia:

- se crea el paquete `jakarta`;
- **Jakarta EE 9** introduce el cambio de nombre sin añadir nuevas funciones;
- antes: `javax.servlet.*`;
- ahora: `jakarta.servlet.*`.

La documentación antigua puede seguir usando `javax`, pero en entornos actuales se trabaja con `jakarta`.

5.10 Ideas que no se deben confundir

No confundir	Diferencia
Servlet vs HTML	El servlet es código Java del servidor; HTML es la respuesta o vista.
Servlet vs navegador	El servlet vive en el contenedor web, no en el navegador.
<code>request</code> vs <code>response</code>	<code>request</code> es lo que llega del cliente; <code>response</code> es lo que se devuelve.
Parámetro vs atributo	Parámetro es configuración o dato textual enviado; atributo es objeto compartido en ejecución.
<code>forward()</code> vs <code>sendRedirect()</code>	<code>forward()</code> reenvía internamente; <code>sendRedirect()</code> crea nueva petición del cliente.

No confundir	Diferencia
Sesión vs cookie	La sesión se mantiene en servidor; la cookie suele transportar el identificador.

5.11 Plantilla compacta de examen

Si piden una aplicación web con servlets, JSP y `web.xml`, una estructura MVC típica es:

```
src/main/java/daw/
├─ model/UsuarioBean.java
└─ web/
    ├─ InicioServlet.java
    ├─ LoginServlet.java
    ├─ ListadoServlet.java
    └─ LogoutServlet.java

src/main/webapp/WEB-INF/
├─ jsp/login.jsp
├─ jsp/panel.jsp
└─ web.xml
```

Roles:

Elemento	Papel
<code>UsuarioBean</code>	Bean de datos: constructor vacío, atributos privados, getters/setters.
<code>InicioServlet</code>	<code>doGet()</code> : hace <code>forward()</code> a <code>login.jsp</code> .
<code>LoginServlet</code>	<code>doPost()</code> : lee formulario con <code>getParameter()</code> , valida, guarda usuario en <code>session</code> o vuelve al login con atributo de error.
<code>ListadoServlet</code>	<code>doGet()</code> : comprueba sesión, carga datos en <code>request</code> y hace <code>forward()</code> a <code>panel.jsp</code> .
<code>LogoutServlet</code>	<code>doGet()</code> : recupera sesión con <code>getSession(false)</code> , invalida y redirige a inicio.
<code>web.xml</code>	Mapea URLs como <code>/inicio</code> , <code>/login</code> , <code>/listado</code> , <code>/logout</code> y puede configurar <code>session-timeout</code> .

Flujo que debes saber explicar:

- `/inicio` entra en `InicioServlet`.
- `InicioServlet` hace `forward()` a `login.jsp`.
- El formulario envía `POST` a `/login`.
- `LoginServlet` valida.
- Si es correcto, guarda `UsuarioBean` en `session` y hace `sendRedirect()` a `/listado`.

6. `ListadoServlet` comprueba sesión, carga datos en `request` y hace `forward()` a `panel.jsp`.
7. `panel.jsp` muestra usuario y datos.
8. `LogoutServlet` invalida sesión y redirige a `/inicio`.

Patrones de código que suelen bastar:

```
request.getRequestDispatcher("/WEB-INF/jsp/login.jsp").forward(request, response);
String nombre = request.getParameter("nombre");
request.getSession().setAttribute("usuario", usuario);
response.sendRedirect(request.getContextPath() + "/listado");
HttpSession session = request.getSession(false);
if (session != null) session.invalidate();
```

La parte JSP de este flujo queda desarrollada en el tema `6. JSP`.

6. JSP

6.1 Introducción a JSP

JSP (Java Server Pages) son páginas HTML con líneas de código adicional que se ejecutan en el servidor para generar contenido dinámico.

Ideas clave:

- se traducen automáticamente a **servlets**;
- después de traducirse se compilan y se ejecutan como servlets;
- si se modifica la JSP, el contenedor vuelve a traducirla;
- ayudan a separar presentación y lógica de negocio;
- la lógica de negocio debe quedar encapsulada en servlets, JavaBeans o clases del modelo, no mezclada en la vista.

Componentes JSP

Una JSP se parece a HTML, pero puede contener componentes especiales, muchos delimitados por

`<% ... %>`.

Componente	Sintaxis	Uso	Traducción al servlet
Comentario JSP	<code><!-- ... --></code>	Comentario que no se envía al cliente.	No aparece en la respuesta.
Declaración	<code><%! ... %></code>	Define variables y métodos.	Fuera de <code>_jspService()</code> .
Expresión	<code><%= ... %></code>	Evalúa Java y escribe el resultado.	Se inserta como salida.
Scriptlet	<code><% ... %></code>	Código Java arbitrario. Puede declarar variables, pero no métodos.	Dentro de <code>_jspService()</code> .
Directiva	<code><%@ ... %></code>	Instrucciones de compilación/traducción.	Modifican el servlet resultante.
Acción JSP	<code><jsp:accion ... /></code>	Control de flujo y acceso a JavaBeans.	Se evalúa en cada petición.

Directivas

Las directivas se procesan cuando la página se traduce a servlet. No devuelven contenido, pero afectan a la estructura resultante.

Directiva	Uso
<code>page</code>	Define características globales de la JSP.
<code>include</code>	Inserta contenido de ficheros externos en tiempo de traducción.
<code>taglib</code>	Declara librerías de etiquetas estándar o propias.

Directiva `page`

Atributo	Significado
<code>language</code>	Lenguaje de scripting; Java por defecto.
<code>extends</code>	Superclase del servlet generado.
<code>import</code>	Lista de clases o paquetes importados.
<code>session</code>	Indica si la JSP participa en sesiones HTTP.
<code>buffer</code>	Tamaño del buffer; por defecto 8 KB.
<code>autoFlush</code>	Si el buffer se vacía automáticamente.
<code>isThreadSafe</code>	Si el contenedor puede despachar varios hilos simultáneamente.
<code>info</code>	Texto devuelto por <code>getServletInfo()</code> .
<code>errorPage</code>	URL de la página a usar en caso de error.
<code>isErrorPage</code>	Si es página de error y puede acceder a <code>exception</code> .
<code>contentType</code>	Tipo MIME de la respuesta; por defecto se cita <code>text/html; charset=ISO-8859-1</code> .
<code>pageEncoding</code>	Codificación de la página.
<code>isELIgnored</code>	Si se ignoran expresiones EL.

Acciones JSP

Las acciones usan sintaxis XML y se reevalúan cada vez que se accede a la página.

Acción	Uso
<code>jsp:forward</code>	Reenvía <code>request</code> y <code>response</code> a otro recurso.
<code>jsp:include</code>	Incluye otro recurso en la página.
<code>jsp:param</code>	Añade parámetros a <code>forward</code> o <code>include</code> .
<code>jsp:useBean</code>	Localiza o instancia un JavaBean.

Acción	Uso
<code>jsp:setProperty</code>	Modifica una propiedad de un JavaBean.
<code>jsp:getProperty</code>	Recupera una propiedad de un JavaBean.
<code>jsp:plugin</code>	Incrusta componentes como applets.
<code>jsp:fallback</code>	Mensaje alternativo para <code>jsp:plugin</code> .

Comparación de includes:

Directiva <code>include</code>	Acción <code>jsp:include</code>
Incluye en el momento de traducción.	Incluye en el momento de la llamada.
Mejor para páginas estáticas.	Mejor para páginas dinámicas.
No permite parámetros.	Permite parámetros.
Integra el contenido en el servlet generado.	Evalúa el recurso al ejecutar la petición.

6.2 Objetos implícitos, atributos y reenvío

Los **objetos implícitos** son objetos declarados e inicializados por el contenedor, disponibles dentro de `_jspService()` sin crearlos manualmente.

El acceso se organiza alrededor de `pageContext`, que se obtiene mediante `getPageContext()` de `JspFactory`.

Objeto	Clase	Uso	Método desde <code>pageContext</code>
<code>out</code>	<code>JspWriter</code>	Almacena la respuesta enviada al cliente.	<code>getOut()</code>
<code>request</code>	<code>HttpServletRequest</code>	Datos enviados por el cliente.	<code>getRequest()</code>
<code>response</code>	<code>HttpServletResponse</code>	Datos enviados por el servidor.	<code>getResponse()</code>
<code>session</code>	<code>HttpSession</code>	Identificación del usuario entre páginas.	<code>getSession()</code>
<code>application</code>	<code>ServletContext</code>	Comunicación con el contenedor y datos globales.	<code>getServletContext()</code>
<code>config</code>	<code>ServletConfig</code>	Configuración de inicialización.	<code>getServletConfig()</code>

Objeto	Clase	Uso	Método desde <code>pageContext</code>
<code>exception</code>	<code>JspException</code> / excepción	Información de excepción en páginas de error.	Disponible si <code>isErrorPage=true</code> .
<code>page</code>	<code>Object</code>	Servlet obtenido de la JSP.	Referencia al servlet.
<code>pageContext</code>	<code>PageContext</code>	Manejo de atributos en <code>page</code> , <code>request</code> , <code>session</code> , <code>application</code> .	<code>getPageContext()</code>

Atributos

Los atributos en JSP son análogos a los de servlets, pero añaden el ámbito **page**.

Ámbito	Servlet	JSP
Aplicación	<code>getServletContext().get/setAttribute()</code>	<code>application.get/setAttribute()</code>
Petición	<code>request.get/setAttribute()</code>	<code>request.get/setAttribute()</code>
Sesión	<code>request.getSession().get/setAttribute()</code>	<code>session.get/setAttribute()</code>
Página	No existe	<code>pageContext.get/setAttribute()</code>

Constantes de `pageContext`:

- `APPLICATION_SCOPE`;
- `PAGE_SCOPE`;
- `REQUEST_SCOPE`;
- `SESSION_SCOPE`.

Métodos útiles:

- `pageContext.findAttribute(nombre)`: busca en `page`, `request`, `session`, `application`;
- `getAttributeNamesInScope(scope)`: lista atributos de un ámbito.

Reenvío de petición

Para delegar procesamiento se puede usar `RequestDispatcher` desde un servlet o acciones JSP desde una JSP.

Operación	Efecto
<code>forward(request, response)</code>	Cede el control a otro recurso con la misma petición y respuesta.
<code>include(request, response)</code>	Inserta la salida de otro recurso sin perder el control.

Operación	Efecto
<code>jsp:forward</code>	Equivalente JSP para reenviar.
<code>jsp:include</code>	Equivalente JSP para incluir.

Como se reutiliza la misma `request`, los datos se pueden pasar con `request.setAttribute(...)` antes del `forward`.

6.3 JavaBeans

Los scriptlets tienen dos problemas principales:

- al crecer la JSP, se vuelven largos y difíciles de mantener;
- la funcionalidad queda ligada a una JSP concreta y se reutiliza mal.

La solución es mover datos y lógica sencilla a **JavaBeans**.

Ventajas:

- reutilización de código;
- mejor separación entre diseñador y desarrollador;
- JSP más orientada a presentación.

Un **JavaBean** es una clase Java que mantiene datos mediante propiedades accesibles por getters y setters.

Convenciones:

- constructor sin argumentos;
- atributos privados;
- getters y setters con nomenclatura estándar;
- implementación de `Serializable`.

Acciones para JavaBeans

Acción	Uso
<code>jsp:useBean</code>	Localiza un bean y lo instancia si no existe.
<code>jsp:setProperty</code>	Asigna una propiedad.
<code>jsp:getProperty</code>	Recupera una propiedad.

`jsp:useBean` también guarda el objeto como atributo en un ámbito.

Ámbitos habituales:

- `page`;
- `request`;
- `session`;
- `application`.

Comportamiento de `jsp:useBean`:

1. busca el bean por `id` en el ámbito indicado;
2. si existe, lo reutiliza;
3. si no existe, lo instancia;
4. lo guarda como atributo;
5. si tiene cuerpo, el cuerpo solo se ejecuta cuando el bean se crea en ese momento.

Beans creados en servlets

Patrón MVC recomendado:

1. el servlet crea y rellena el bean;
2. lo guarda como atributo en `request` o `session`;
3. hace `forward()` a la JSP;
4. la JSP solo muestra datos con EL, JSTL o acciones JSP.

Ejemplo conceptual:

```
LoginBean bean = new LoginBean();
bean.setNombre(nombre);
request.setAttribute("pwdBean", bean);
request.getRequestDispatcher("welcome.jsp").forward(request, response);
```

En JSP puede recuperarse con `jsp:useBean` indicando el mismo `scope`, o más modernamente con EL: `${pwdBean.nombre}`.

Referencias polimórficas

Si el bean se instancia directamente, basta con `class`.

Si se quiere declarar una referencia de un tipo y crear una clase concreta, se combinan:

Atributo	Uso
<code>type</code>	Tipo de la referencia visible.

Atributo	Uso
<code>class</code>	Clase concreta que se instancia.

Esto evita problemas cuando el tipo de referencia es abstracto. Si se intenta instanciar una clase abstracta con `class`, puede producirse `InstantiationException`.

Formularios y JavaBeans

Los datos de formulario se pueden copiar a un bean:

Caso	Forma
Campo y propiedad no coinciden	<code>property="name" param="userName"</code> .
Usar expresión Java	<code>value="<%= request.getParameter(...) %>"</code> .
Campo y propiedad coinciden	<code>property="name"</code> sin <code>param</code> .
Copiar todas las coincidencias	<code>property="*"</code> .

`property="*"` intenta rellenar todas las propiedades para las que exista un parámetro de petición con el mismo nombre.

6.4 JSP Expression Language

JSP Expression Language (EL) proporciona una sintaxis compacta para manejar JavaBeans, arrays, listas y mapas almacenados como atributos.

Para reconocer EL puede usarse:

```
<%@ page isELIgnored="false" %>
```

Ventajas:

- sintaxis compacta;
- acceso cómodo a propiedades anidadas;
- acceso a JavaBeans y colecciones;
- acceso a parámetros, cabeceras, cookies y parámetros de inicialización;
- permite comparaciones y cálculos.

Limitación:

- EL está pensada para **leer** datos, no para instanciar o modificar JavaBeans.

Sintaxis y búsqueda por ámbitos

Sintaxis:

```
${objeto.propiedad}
${lista[0]}
${mapa["clave"]}
```

Si `objeto` no es implícito de EL, se busca como atributo en este orden:

1. `page`;
2. `request`;
3. `session`;
4. `application`.

Para evitar conflictos se puede indicar el ámbito:

```
${pageScope.nombre}
${requestScope.error}
${sessionScope.usuario.nombre}
${applicationScope.valor}
```

Objetos implícitos EL

Objeto EL	Uso
<code>pageScope</code>	Atributos de la página JSP actual.
<code>requestScope</code>	Atributos de la petición actual.
<code>sessionScope</code>	Atributos de la sesión.
<code>applicationScope</code>	Atributos globales de aplicación.
<code>param</code>	Parámetro simple de URL o formulario.
<code>paramValues</code>	Parámetros con varios valores, como <code>select multiple</code> .
<code>header</code>	Cabecera HTTP.
<code>headerValues</code>	Cabeceras HTTP multivalor.
<code>cookie</code>	Cookies del usuario.
<code>initParam</code>	Parámetros globales de inicialización configurados en XML.
<code>pageContext</code>	Acceso a ámbitos, atributos y otros objetos implícitos.

Ejemplos:

```

${param.name}
${paramValues.food[1]}
${cookie.asignatura.value}
${initParam.miEmail}

```

Operadores

Tipo	Operadores		
Acceso	<code>.</code> y <code>[indice/clave]</code> .		
Aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> o <code>div</code> , <code>%</code> o <code>mod</code> .		
Relacionales	<code>==</code> o <code>eq</code> , <code>!=</code> o <code>ne</code> , <code><</code> o <code>lt</code> , <code>></code> o <code>gt</code> , <code><=</code> o <code>le</code> , <code>>=</code> o <code>ge</code> .		
Lógicos	<code>&&</code> o <code>and</code> , <code>`</code>		<code>o or</code> , <code>!</code> o <code>not</code> .
Vacío	<code>empty x: true</code> si <code>x</code> es <code>null</code> , string vacío, array/list/map vacío.		
Ternario	<code>x ? y : z</code> .		

Uso dentro de MVC:

```

request.setAttribute("pwdBean", bean);
request.getRequestDispatcher("welcome.jsp").forward(request, response);

```

```

<h2>Bienvenido ${pwdBean.nombre}</h2>

```

La idea moderna es: **el servlet prepara, la JSP muestra.**

6.5 JSTL

JSTL (JavaServer Pages Standard Tag Library) amplía JSP con etiquetas estándar para eliminar scriptlets.

Para usarla:

- incluir `jstl.jar` en `WEB-INF/lib`;
- declarar la librería con `taglib`.

Ejemplo:

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

```

Ventajas:

- permite marcas más claras;
- es más sencilla, robusta y portable;
- permite crear marcas a medida;
- ayuda a que la vista no contenga Java embebido.

Librerías JSTL 1.2

Librería	URI	Prefijo	Uso
Core	<code>http://java.sun.com/jsp/jstl/core</code>	<code>c</code>	Flujo, iteraciones, URLs y operaciones generales.
Formatting	<code>http://java.sun.com/jsp/jstl/fmt</code>	<code>fmt</code>	Internacionalización y fechas/números.
SQL	<code>http://java.sun.com/jsp/jstl/sql</code>	<code>sql</code>	Bases de datos relacionales.
XML	<code>http://java.sun.com/jsp/jstl/xml</code>	<code>x</code>	XML, flujo y transformaciones.
Functions	<code>http://java.sun.com/jsp/jstl/functions</code>	<code>fn</code>	Cadenas y funciones auxiliares.

Librería `core`

Etiqueta	Atributos principales	Resultado
<code>c:out</code>	<code>value</code>	Muestra una expresión.
<code>c:set</code>	<code>var</code> , <code>scope</code> , <code>value</code>	Evalúa y guarda valor como atributo.
<code>c:remove</code>	<code>var</code>	Elimina un atributo.
<code>c:catch</code>	<code>var</code>	Captura excepción en una variable.
<code>c:forEach</code>	<code>var</code> , <code>begin</code> , <code>end</code> , <code>items</code>	Itera rango o colección.
<code>c:forTokens</code>	<code>var</code> , <code>items</code> , <code>delims</code>	Itera tokens de una cadena.
<code>c:if</code>	<code>test</code>	Muestra contenido si la condición es cierta.
<code>c:choose</code>	-	Condicional múltiple.
<code>c:when</code>	<code>test</code>	Rama condicional de <code>choose</code> .
<code>c:otherwise</code>	-	Rama por defecto.
<code>c:import</code>	<code>url</code>	Importa contenido de otro recurso.
<code>c:url</code>	<code>value</code> , <code>var</code>	Construye una URL, normalmente segura para contexto/sesión.
<code>c:redirect</code>	<code>url</code>	Redirige a otra URL.
<code>c:param</code>	<code>name</code> , <code>value</code>	Añade parámetros a <code>c:url</code> , <code>c:redirect</code> o <code>c:import</code> .

Ejemplos conceptuales: 1

```
<c:out value="\${expresion}" />
<c:set var="nombre" scope="request" value="\${expresion}" />
<c:forEach var="i" items="\${array}">\${i}</c:forEach>
<c:if test="\${j gt 0}">positivo</c:if>
<c:url var="logout" value="/logout" />
```

`c:out` es preferible a imprimir texto sin escapar cuando el valor puede contener caracteres HTML/XML.

Otras librerías

`fmt`:

- internacionalización: `fmt:message`, `fmt:setLocale`, `fmt:bundle`, `fmt:setBundle`, `fmt:param`, `fmt:requestEncoding`;
- formato: `fmt:timeZone`, `fmt:setTimeZone`, `fmt:formatNumber`, `fmt:parseNumber`, `fmt:parseDate`.

`sql`:

- `sql:query`;
- `sql:update`;
- `sql:setDataSource`;
- `sql:param`;
- `sql:dateParam`.

Existe para teoría, pero en aplicaciones bien estructuradas el acceso real a datos debe estar en el modelo o capa de servicio.

`xml`:

- acciones básicas: `x:parse`, `x:out`, `x:set`;
- control de flujo: `x:if`, `x:choose`, `x:when`, `x:otherwise`, `x:forEach`;
- transformaciones: `x:transform`, `x:param`.

`fn`:

- `fn:length`, `fn:toUpperCase`, `fn:toLowerCase`;
- `fn:substring`, `fn:substringAfter`, `fn:substringBefore`;
- `fn:trim`, `fn:replace`, `fn:indexOf`;
- `fn:startsWith`, `fn:endsWith`;
- `fn:contains`, `fn:containsIgnoreCase`;
- `fn:split`, `fn:join`, `fn:escapeXML`.

Evolución práctica

La evolución recomendada en JSP es:

1. scriptlets: mezclan Java y HTML;
2. JavaBeans: separan datos;
3. EL: lee datos con sintaxis compacta;
4. JSTL: añade bucles, condicionales y utilidades sin Java embebido.

6.6 Preguntas y plantilla de examen

Definiciones breves:

Concepto	Definición
JSP	Página HTML con etiquetas y fragmentos Java que el servidor traduce a servlet para generar contenido dinámico.
JavaBean	Clase reutilizable con constructor vacío, atributos privados, getters/setters y normalmente <code>Serializable</code> .
EL	Lenguaje de expresiones <code>\${...}</code> para leer atributos, beans, colecciones, parámetros, cabeceras, cookies y contexto.
JSTL	Librería estándar de etiquetas que sustituye scriptlets con etiquetas para flujo, iteración, formato, XML, SQL y funciones.

Relación MVC:

- servlet = controlador;
- JSP = vista;
- JavaBean = datos/modelo simple.

JSP del ejercicio Servlet/JSP

Este bloque encaja con la plantilla del tema `5. Servlets`.

`login.jsp` debe demostrar:

- directiva `page` con `contentType` y `pageEncoding`;
- directiva `taglib` para JSTL core;
- formulario que envía `POST` a `/login`;
- lectura de error con EL o JSTL: `${error}`;
- uso de `${pageContext.request.contextPath}` para construir rutas.

Patrón:

```
<c:if test="${not empty error}">
  <p>${error}</p>
```

```

</c:if>

<form action="${pageContext.request.contextPath}/login" method="post">
  <input name="nombre">
  <input name="pwd" type="password">
</form>

```

`panel.jsp` debe demostrar:

- acceso a un bean guardado en sesión: `${sessionScope.usuario.nombre}`;
- iteración de una lista con `c:forEach`;
- salida segura con `c:out`;
- enlace de cierre de sesión.

Patrón:

```

<h1>Bienvenido ${sessionScope.usuario.nombre}</h1>

<c:forEach var="asig" items="${asignaturas}">
  <li><c:out value="${asig}" /></li>
</c:forEach>

<a href="${pageContext.request.contextPath}/logout">Cerrar sesión</a>

```

Si en examen piden varios servlets, JSP y `web.xml`, la explicación queda:

- tema 5: servlets, `request/session`, `forward`, `redirect`, `web.xml`;
- tema 6: JSP, JavaBeans, EL y JSTL como capa de vista.

GlosarioMasticado

1. HTML

HTML sirve para definir la **estructura** de una página web.

1.1. Estructura básica del documento

Elemento	Uso	Ejemplo
<code><!DOCTYPE html></code>	Indica que el documento usa HTML5.	<code><!DOCTYPE html></code>
<code><html></code>	Raíz del documento.	<code><html>...</html></code>
<code><head></code>	Contiene metadatos, título, CSS, scripts.	<code><head>...</head></code>
<code><body></code>	Contiene lo visible de la página.	<code><body>...</body></code>
<code><meta></code>	Define metadatos, como codificación.	<code><meta charset="UTF-8"></code>
<code><title></code>	Título de la pestaña del navegador.	<code><title>Mi página</title></code>
<code><link></code>	Enlaza recursos externos, normalmente CSS.	<code><link rel="stylesheet" href="style.css"></code>
<code><script></code>	Inserta o enlaza JavaScript.	<code><script src="codigo.js"></script></code>
<code><style></code>	CSS interno dentro del HTML.	<code><style>p { color: red; }</style></code>

1.2. Texto y formato

Elemento	Uso	Ejemplo
<code><h1></code> a <code><h6></code>	Encabezados, de mayor a menor importancia.	<code><h1>Título</h1></code>
<code><p></code>	Párrafo.	<code><p>Texto</p></code>
<code>
</code>	Salto de línea. Es elemento vacío.	Hola Adiós
<code></code>	Texto importante, normalmente en negrita.	<code>Importante</code>
<code></code>	Negrita visual, sin tanta carga semántica.	<code>Texto</code>

Elemento	Uso	Ejemplo
<code></code>	Énfasis semántico, normalmente cursiva.	<code>Atención</code>
<code><i></code>	Cursiva visual o término especial.	<code><i>status quo</i></code>
<code><mark></code>	Texto resaltado.	<code><mark>clave</mark></code>
<code><blockquote></code>	Cita larga.	<code><blockquote>Texto citado</blockquote></code>
<code><q></code>	Cita corta en línea.	<code><q>Hola</q></code>
<code><pre></code>	Texto preformateado, respeta espacios y saltos.	<code><pre>línea 1\nlínea 2</pre></code>
<code><time></code>	Fecha u hora semántica.	<code><time datetime="2026-05-17">17 mayo</time></code>

Diferencia importante:

```

<b>negrita visual</b>
<strong>importancia semántica</strong>

<i>cursiva visual</i>
<em>énfasis semántico</em>

```

1.3. Contenedores y estructura semántica

Elemento	Uso	Ejemplo
<code><div></code>	Contenedor genérico de bloque.	<code><div class="caja">...</div></code>
<code></code>	Contenedor genérico en línea.	<code>texto</code>
<code><header></code>	Cabecera de página o sección.	<code><header>...</header></code>
<code><footer></code>	Pie de página o sección.	<code><footer>...</footer></code>
<code><main></code>	Contenido principal.	<code><main>...</main></code>
<code><section></code>	Sección temática.	<code><section>Noticias</section></code>
<code><article></code>	Contenido independiente.	<code><article>Post</article></code>
<code><aside></code>	Contenido lateral o complementario.	<code><aside>Publicidad</aside></code>
<code><nav></code>	Zona de navegación.	<code><nav>Inicio</nav></code>

Idea rápida:

```
<div>
```

sirve para agrupar sin significado semántico especial.

```
<section>, <article>, <nav>, <main>
```

aportan significado a la estructura.

1.4. Listas

Elemento	Uso	Ejemplo
<code></code>	Lista desordenada.	<code>Uno</code>
<code></code>	Lista ordenada.	<code>Uno</code>
<code></code>	Elemento de lista.	<code>Elemento</code>
<code><dl></code>	Lista de definición.	<code><dl>...</dl></code>
<code><dt></code>	Término.	<code><dt>HTML</dt></code>
<code><dd></code>	Definición.	<code><dd>Lenguaje de marcado</dd></code>

Ejemplo:

```
<dl>
  <dt>HTML</dt>
  <dd>Lenguaje para estructurar páginas web.</dd>
</dl>
```

1.5. Enlaces e imágenes

Elemento	Uso	Ejemplo
<code><a></code>	Enlace.	<code>Ir</code>
<code></code>	Imagen. Elemento vacío.	<code></code>
<code><picture></code>	Imágenes adaptativas.	<code><picture>...</picture></code>
<code><source></code>	Fuente alternativa para imagen, audio o video.	<code><source src="video.mp4" type="video/mp4"></code>
<code><iframe></code>	Inserta otra página dentro.	<code><iframe src="pagina.html"></iframe></code>

Ejemplo de imagen:

```

```

`alt` es importante porque ofrece texto alternativo si la imagen no carga o para accesibilidad.

1.6. Formularios

Elemento	Uso	Ejemplo
<code><form></code>	Formulario.	<code><form action="login" method="post"></code>
<code><input></code>	Campo de entrada.	<code><input type="text" name="usuario"></code>
<code><label></code>	Etiqueta asociada a un campo.	<code><label for="email">Email</label></code>
<code><textarea></code>	Campo de texto largo.	<code><textarea name="mensaje"></textarea></code>
<code><select></code>	Lista desplegable.	<code><select name="ciudad">...</select></code>
<code><datalist></code>	Opciones sugeridas para un input.	<code><datalist id="opciones">...</datalist></code>
<code><fieldset></code>	Agrupar campos.	<code><fieldset>...</fieldset></code>
<code><legend></code>	Título de un fieldset.	<code><legend>Datos personales</legend></code>
<code><button></code>	Botón.	<code><button type="submit">Enviar</button></code>

Ejemplo:

```
<form action="RegistroServlet" method="post">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre">

  <input type="hidden" name="accion" value="registro">

  <button type="submit">Enviar</button>
</form>
```

Campos importantes:

```
<input type="submit">
```

envía el formulario.

```
<input type="button">
```

no hace nada por sí solo; necesita JavaScript.

```
<input type="hidden">
```

envía datos ocultos al usuario, pero visibles/modificables desde el navegador.

1.7. Tablas

Elemento	Uso	Ejemplo
<code><table></code>	Tabla.	<code><table>...</table></code>
<code><caption></code>	Título de tabla.	<code><caption>Usuarios</caption></code>
<code><tr></code>	Fila.	<code><tr>...</tr></code>
<code><th></code>	Celda de cabecera.	<code><th>Nombre</th></code>
<code><td></code>	Celda normal.	<code><td>Ana</td></code>

Ejemplo:

```
<table>
  <caption>Usuarios</caption>
  <tr>
    <th>Nombre</th>
    <th>Edad</th>
  </tr>
  <tr>
    <td>Ana</td>
    <td>24</td>
  </tr>
</table>
```

1.8. Multimedia

Elemento	Uso	Ejemplo
<code><audio></code>	Audio.	<code><audio controls>...</audio></code>
<code><video></code>	Video.	<code><video controls>...</video></code>
<code><source></code>	Fuente de audio/video.	<code><source src="audio.mp3" type="audio/mpeg"></code>
<code><track></code>	Subtítulos.	<code><track src="sub.vtt" kind="subtitles"></code>

Elemento	Uso	Ejemplo
<code><canvas></code>	Área para dibujar con JavaScript.	<code><canvas id="lienzo"></canvas></code>

Ejemplo:

```
<video controls>
  <source src="video.mp4" type="video/mp4">
  Tu navegador no soporta video.
</video>
```

2. CSS

CSS sirve para definir el **aspecto visual** de los elementos HTML.

2.1. Selectores básicos

Selector	Uso	Ejemplo
<code>p</code>	Selecciona etiquetas <code><p></code> .	<code>p { color: red; }</code>
<code>.clase</code>	Selecciona elementos con clase.	<code>.caja { border: 1px solid; }</code>
<code>#id</code>	Selecciona elemento con id.	<code>#menu { background: black; }</code>
<code>p.clase1.clase2</code>	<code><p></code> con ambas clases.	<code>p.activo.rojo { color: red; }</code>
<code>a[href*="usc"]</code>	Atributo que contiene texto.	<code>a[href*="usc"] { color: blue; }</code>

Importante:

```
p.clase1.clase2
```

no exige que tenga solo esas dos clases. Exige que tenga **al menos** esas dos.

2.2. Texto y fuentes

Propiedad	Uso	Ejemplo
<code>color</code>	Color del texto.	<code>color: red;</code>
<code>font-family</code>	Fuente.	<code>font-family: Arial, sans-serif;</code>
<code>font-size</code>	Tamaño.	<code>font-size: 20px;</code>

Propiedad	Uso	Ejemplo
<code>font-style</code>	Cursiva.	<code>font-style: italic;</code>
<code>font-weight</code>	Grosor.	<code>font-weight: bold;</code>
<code>line-height</code>	Altura de línea.	<code>line-height: 1.5;</code>
<code>text-align</code>	Alineación.	<code>text-align: center;</code>
<code>text-decoration</code>	Subrayado, tachado.	<code>text-decoration: underline;</code>
<code>text-transform</code>	Mayúsculas/minúsculas.	<code>text-transform: uppercase;</code>

Ejemplo:

```
h1 {
  font-family: Georgia, serif;
  font-size: 32px;
  text-align: center;
}
```

2.3. Modelo de caja

Propiedad	Uso	Ejemplo
<code>width</code>	Ancho.	<code>width: 200px;</code>
<code>height</code>	Alto.	<code>height: 100px;</code>
<code>margin</code>	Espacio exterior.	<code>margin: 20px;</code>
<code>padding</code>	Espacio interior.	<code>padding: 10px;</code>
<code>border</code>	Borde.	<code>border: 2px solid black;</code>
<code>border-radius</code>	Bordes redondeados.	<code>border-radius: 8px;</code>
<code>box-sizing</code>	Cómo se calcula el tamaño.	<code>box-sizing: border-box;</code>
<code>box-shadow</code>	Sombra.	<code>box-shadow: 0 4px 8px gray;</code>

Ejemplo:

```
.caja {
  width: 200px;
  padding: 20px;
  border: 4px solid black;
  box-sizing: border-box;
}
```

`box-sizing: border-box` hace que el ancho incluya padding y borde.

2.4. Fondos

Propiedad	Uso	Ejemplo
<code>background</code>	Fondo abreviado.	<code>background: red;</code>
<code>background-color</code>	Color de fondo.	<code>background-color: yellow;</code>
<code>background-image</code>	Imagen de fondo.	<code>background-image: url("fondo.jpg");</code>
<code>background-position</code>	Posición.	<code>background-position: center;</code>
<code>background-repeat</code>	Repetición.	<code>background-repeat: no-repeat;</code>

Ejemplo:

```
body {  
  background-color: #f5f5f5;  
}
```

2.5. Display y flujo

Propiedad/valor	Uso	Ejemplo
<code>display: block</code>	Elemento ocupa línea completa.	<code>display: block;</code>
<code>display: inline</code>	Elemento fluye en línea.	<code>display: inline;</code>
<code>display: inline-block</code>	En línea, pero permite ancho/alto.	<code>display: inline-block;</code>
<code>display: flex</code>	Activa flexbox.	<code>display: flex;</code>
<code>display: grid</code>	Activa grid.	<code>display: grid;</code>

Ejemplo:

```
.contenedor {  
  display: flex;  
}
```

2.6. Float y clear

Propiedad	Uso	Ejemplo
<code>float</code>	Hace flotar a izquierda/derecha.	<code>float: left;</code>
<code>clear</code>	Evita colocarse junto a flotantes.	<code>clear: both;</code>

Ejemplo:

```
img {  
  float: left;  
  margin-right: 10px;  
}  
  
h2 {  
  clear: both;  
}
```

`clear: both` significa que el elemento baja por debajo de floats anteriores.

2.7. Position

Valor	Uso
<code>static</code>	Valor normal. No acepta <code>top</code> , <code>left</code> , etc.
<code>relative</code>	Se mueve respecto a su posición original.
<code>absolute</code>	Sale del flujo y se posiciona respecto al ancestro posicionado más cercano.
<code>fixed</code>	Se posiciona respecto a la ventana.
<code>sticky</code>	Mezcla relative y fixed según scroll.

Ejemplo:

```
.contenedor {  
  position: relative;  
}  
  
.hijo {  
  position: absolute;  
  top: 0;  
  right: 0;  
}
```

Importante:

```
right: 200px;
```

con `position: relative` mueve el elemento hacia la **izquierda**, no hacia la derecha.

2.8. Flexbox

Propiedad	Uso
<code>display: flex</code>	Activa flexbox.
<code>flex-direction</code>	Dirección: fila o columna.
<code>flex-wrap</code>	Permite saltar línea.
<code>flex-flow</code>	Abreviatura de direction + wrap.
<code>justify-content</code>	Alinea en eje principal.
<code>align-items</code>	Alinea en eje cruzado.
<code>align-content</code>	Alinea varias líneas.

Ejemplo:

```
.contenedor {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Chuleta:

```
justify-content → eje principal  
align-items     → eje cruzado
```

2.9. Grid

Propiedad	Uso
<code>display: grid</code>	Activa grid.
<code>grid-template-columns</code>	Define columnas.
<code>grid-template-rows</code>	Define filas.

Propiedad	Uso
<code>grid-template-areas</code>	Define áreas.
<code>grid-area</code>	Asigna un elemento a un área.
<code>grid-column</code>	Posición en columnas.
<code>grid-row</code>	Posición en filas.
<code>gap</code> / <code>grid-gap</code>	Separación entre celdas.

Ejemplo:

```
.layout {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-areas:
    "header header"
    "aside main"
    "footer footer";
}

header {
  grid-area: header;
}
```

3. JavaScript DOM

JavaScript permite interactuar con el DOM, crear elementos, modificar texto, estilos y eventos.

3.1. Selección de elementos

Método	Qué devuelve	Ejemplo
<code>getElementById()</code>	Un elemento.	<code>document.getElementById("caja")</code>
<code>getElementsByClassName()</code>	HTMLCollection.	<code>document.getElementsByClassName("item")</code>
<code>getElementsByTagName()</code>	HTMLCollection.	<code>document.getElementsByTagName("p")</code>
<code>getElementsByName()</code>	NodeList.	<code>document.getElementsByName("usuario")</code>
<code>querySelector()</code>	Primer elemento que coincide.	<code>document.querySelector(".caja")</code>
<code>querySelectorAll()</code>	NodeList con todos.	<code>document.querySelectorAll("p")</code>

Ejemplo:

```
const titulo = document.querySelector("h1");
titulo.textContent = "Hola";
```

Confuso:

```
querySelectorAll("p")
```

devuelve una `NodeList`, pero solo con los `<p>` encontrados. No mete nodos texto.

```
elemento.childNodes
```

sí puede incluir nodos texto, comentarios y saltos de línea.

3.2. Crear, añadir y borrar nodos

Método	Uso	Ejemplo
<code>createElement()</code>	Crea un elemento.	<code>document.createElement("p")</code>
<code>createTextNode()</code>	Crea nodo de texto.	<code>document.createTextNode("Hola")</code>
<code>appendChild()</code>	Añade un nodo hijo.	<code>padre.appendChild(hijo)</code>
<code>removeChild()</code>	Elimina un hijo.	<code>padre.removeChild(hijo)</code>

Ejemplo:

```
const p = document.createElement("p");
p.textContent = "ETSE USC";

const caja = document.getElementById("miCaja");
caja.appendChild(p);
```

Borrar:

```
const p = document.querySelector("#miCaja p");

if (p !== null) {
  p.parentNode.removeChild(p);
}
```

3.3. Contenido HTML y texto

Propiedad	Uso	Ejemplo
<code>textContent</code>	Texto plano.	<code>p.textContent = "Hola";</code>
<code>innerHTML</code>	HTML interno.	<code>div.innerHTML = "Hola";</code>
<code>nodeValue</code>	Valor de un nodo.	<code>texto.nodeValue</code>

Diferencia:

```
elemento.textContent = "<b>Hola</b>";
```

muestra literalmente `<b>Hola</b>`.

```
elemento.innerHTML = "<b>Hola</b>";
```

crea negrita real.

3.4. Atributos

Método/propiedad	Uso	Ejemplo
<code>getAttribute()</code>	Lee atributo.	<code>a.getAttribute("href")</code>
<code>setAttribute()</code>	Cambia atributo.	<code>img.setAttribute("src", "foto.jpg")</code>
<code>removeAttribute()</code>	Elimina atributo.	<code>input.removeAttribute("disabled")</code>
<code>hasAttribute()</code>	Comprueba si existe.	<code>el.hasAttribute("id")</code>
<code>getAttributeNode()</code>	Devuelve nodo atributo.	<code>el.getAttributeNode("class")</code>
<code>attributes</code>	Lista de atributos.	<code>el.attributes</code>

Ejemplo:

```
const enlace = document.querySelector("a");

if (enlace.hasAttribute("href")) {
  console.log(enlace.getAttribute("href"));
}
```

Confuso:

```
elemento.attributes
```

devuelve un `NamedNodeMap`, no un array normal.

3.5. Nodos y navegación DOM

Propiedad	Uso
<code>childNodes</code>	Hijos incluyendo texto y comentarios.
<code>children</code>	Solo elementos hijos.
<code>firstChild</code>	Primer nodo hijo.
<code>firstElementChild</code>	Primer elemento hijo.
<code>lastChild</code>	Último nodo hijo.
<code>parentNode</code>	Padre.
<code>nextSibling</code>	Siguiente nodo hermano.
<code>previousSibling</code>	Nodo hermano anterior.
<code>nodeName</code>	Nombre del nodo.
<code>nodeType</code>	Tipo de nodo.

Ejemplo:

```
const lista = document.querySelector("ul");

console.log(lista.children);    // solo li
console.log(lista.childNodes); // puede incluir textos/saltos de línea
```

Tipos frecuentes de `nodeType`:

```
1 → nodo elemento
2 → nodo atributo
3 → nodo texto
```

3.6. Estilos desde JavaScript

Propiedad/método	Uso	Ejemplo
<code>style</code>	Modifica estilos inline.	<code>el.style.color = "red"</code>

Propiedad/método	Uso	Ejemplo
<code>getComputedStyle()</code>	Lee estilo final calculado.	<code>getComputedStyle(el)</code>
<code>getPropertyValue()</code>	Lee propiedad CSS.	<code>estilos.getPropertyValue("background-color")</code>

Ejemplo:

```
const caja = document.querySelector(".caja");

caja.style.backgroundColor = "red";

const estilos = window.getComputedStyle(caja);
console.log(estilos.getPropertyValue("background-color"));
```

Importante:

```
getComputedStyle()
```

sirve para leer, no para modificar.

```
elemento.style
```

sirve para modificar estilos inline.

3.7. Eventos

Método/propiedad	Uso	Ejemplo
<code>addEventListener()</code>	Escucha eventos.	<code>btn.addEventListener("click", f)</code>
<code>e.type</code>	Tipo de evento.	<code>"click"</code>
<code>e.target</code>	Elemento donde ocurrió.	<code>e.target</code>
<code>e.preventDefault()</code>	Cancela acción por defecto.	Formulario no se envía.
<code>this</code>	Elemento que ejecuta el listener.	<code>this.style.color = "red"</code>

Ejemplo:

```
const boton = document.querySelector("button");
```

```
boton.addEventListener("click", function(e) {  
  console.log(e.type);  
  console.log(e.target);  
});
```

Bubbling y capturing:

```
elemento.addEventListener("click", funcion, false);
```

bubbling, por defecto.

```
elemento.addEventListener("click", funcion, true);
```

capturing.

3.8. Funciones y argumentos

Elemento	Uso	Ejemplo
<code>function()</code>	Función tradicional.	<code>function saludar() {}</code>
Arrow function	Función compacta.	<code>const f = () => {}</code>
<code>arguments</code>	Lista de argumentos en función tradicional.	<code>arguments[0]</code>
<code>arguments.length</code>	Número de argumentos.	<code>arguments.length</code>

Ejemplo:

```
function sumar() {  
  console.log(arguments.length);  
}  
  
sumar(1, 2, 3);
```

Arrow function no tiene su propio `arguments`.

3.9. Conversión de tipos

Método	Uso	Ejemplo
<code>parseInt()</code>	Convierte a entero.	<code>parseInt("12")</code>

Método	Uso	Ejemplo
<code>parseFloat()</code>	Convierte a decimal.	<code>parseFloat("12.5")</code>
<code>toString()</code>	Convierte a texto.	<code>(123).toString()</code>

Ejemplo:

```
const n = parseFloat("4.5");
console.log(n + 1);
```

4. Expresiones regulares

Sirven para buscar patrones en texto.

4.1. Métodos principales

Método	Uso	Ejemplo
<code>test()</code>	Devuelve <code>true</code> o <code>false</code> .	<code>/a/.test("casa")</code>
<code>exec()</code>	Devuelve coincidencia detallada.	<code>/a/.exec("casa")</code>
<code>match()</code>	Busca coincidencias desde string.	<code>"casa".match(/a/g)</code>
<code>matchAll()</code>	Todas las coincidencias con grupos.	<code>"Ana:20".matchAll(/(\w+):(\d+)/g)</code>
<code>replace()</code>	Reemplaza primera o según regex.	<code>"hola".replace("h", "H")</code>
<code>replaceAll()</code>	Reemplaza todas.	<code>"aaa".replaceAll("a", "b")</code>
<code>search()</code>	Devuelve posición.	<code>"hola".search(/o/)</code>
<code>split()</code>	Divide texto.	<code>"a,b".split(",")</code>

Ejemplo:

```
const regex = /^[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}$/;

console.log(regex.test("a@b.com"));
```

4.2. Flags

Flag	Nombre	Uso
<code>g</code>	global	Busca todas las coincidencias.
<code>i</code>	ignoreCase	Ignora mayúsculas/minúsculas.
<code>m</code>	multiline	<code>^</code> y <code>\$</code> funcionan por líneas.

Ejemplo:

```
const texto = "Hola hola";  
  
console.log(texto.match(/hola/gi));
```

4.3. `match()` VS `matchAll()`

```
"Hola mundo".match(/(Hola) mundo/);
```

devuelve:

```
["Hola mundo", "Hola"]
```

porque no hay `g`.

Con `g`:

```
"Hola mundo Hola tierra".match(/(Hola) \w+/g);
```

devuelve:

```
["Hola mundo", "Hola tierra"]
```

sin grupos.

Con `matchAll()`:

```
const r = "Hola mundo Hola tierra".matchAll(/(Hola) \w+/g);
```

puedes obtener todas las coincidencias y sus grupos.

5. jQuery

jQuery simplifica selección, eventos, estilos, animaciones y DOM.

5.1. Selección y objeto jQuery

Elemento	Uso	Ejemplo
<code>\$()</code>	Selecciona o crea elementos.	<code>\$("p")</code>
<code>\$(this)</code>	Elemento actual como objeto jQuery.	<code>\$(this).css("color","red")</code>
<code>.ready()</code>	Ejecuta al cargar DOM.	<code>\$(document).ready(function(){})</code>

Ejemplo:

```
$(document).ready(function() {  
    $("p").text("Hola");  
});
```

5.2. Contenido y valores

Método	Uso	Ejemplo
<code>.text()</code>	Lee/cambia texto.	<code>\$("p").text("Hola")</code>
<code>.html()</code>	Lee/cambia HTML.	<code>\$("div").html("Hola")</code>
<code>.val()</code>	Valor de input.	<code>\$("input").val()</code>

Ejemplo:

```
const valor = $("#nombre").val();
```

5.3. Estilos y clases

Método	Uso	Ejemplo
<code>.css()</code>	Lee/cambia CSS.	<code>\$("p").css("color","red")</code>
<code>.addClass()</code>	Añade clase.	<code>\$("p").addClass("activo")</code>
<code>.removeClass()</code>	Quita clase.	<code>\$("p").removeClass("activo")</code>
<code>.toggleClass()</code>	Alterna clase.	<code>\$("p").toggleClass("activo")</code>

Método	Uso	Ejemplo
<code>.attr()</code>	Lee/cambia atributo.	<code>\$("img").attr("src","a.png")</code>
<code>.removeAttr()</code>	Elimina atributo.	<code>\$("input").removeAttr("disabled")</code>

Ejemplo:

```

$("div").css({
  backgroundColor: "red",
  width: "100px"
});

```

5.4. DOM

Método	Uso	Ejemplo
<code>.append()</code>	Añade al final.	<code>\$("ul").append("Nuevo")</code>
<code>.prepend()</code>	Añade al principio.	<code>\$("ul").prepend("Primero")</code>
<code>.after()</code>	Inserta después.	<code>\$("p").after("<hr>")</code>
<code>.before()</code>	Inserta antes.	<code>\$("p").before("<hr>")</code>
<code>.remove()</code>	Elimina elemento.	<code>\$("p").remove()</code>
<code>.empty()</code>	Vacía contenido.	<code>\$("div").empty()</code>
<code>.clone()</code>	Clona elemento.	<code>\$("p").clone()</code>
<code>.unwrap()</code>	Quita padre.	<code>\$("span").unwrap()</code>

Ejemplo:

```

$("#miCaja").append("<p>ETSE USC</p>");

```

5.5. Recorrido DOM

Método	Uso
<code>.parent()</code>	Padre directo.
<code>.parents()</code>	Todos los ancestros.
<code>.children()</code>	Hijos directos.
<code>.find()</code>	Descendientes que coinciden.

Método	Uso
<code>.closest()</code>	Ancestro más cercano que coincide.
<code>.siblings()</code>	Hermanos.
<code>.next()</code>	Siguiente hermano.
<code>.prev()</code>	Hermano anterior.
<code>.eq()</code>	Elemento por índice.
<code>.filter()</code>	Filtra colección.
<code>.not()</code>	Excluye.

Ejemplo:

```
$("ul").find("li").eq(0).css("color", "red");
```

5.6. Eventos jQuery

Método	Uso	Ejemplo
<code>.on()</code>	Asigna eventos.	<code>\$("#p").on("click", f)</code>
<code>.preventDefault()</code>	Cancela acción.	<code>e.preventDefault()</code>
<code>.stopPropagation()</code>	Para propagación.	<code>e.stopPropagation()</code>

Ejemplo con varios eventos:

```
$("#h2").on("click mouseover", function(e) {
    console.log(e.type);
});
```

En jQuery puedes poner varios eventos separados por espacios.

5.7. Animaciones y efectos

Método	Uso
<code>.hide()</code>	Ocultar.
<code>.show()</code>	Muestra.
<code>.toggle()</code>	Alternar visible/oculto.
<code>.fadeIn()</code>	Aparece con fundido.

Método	Uso
<code>.fadeOut()</code>	Desaparece con fundido.
<code>.fadeToggle()</code>	Alterna fundido.
<code>.slideDown()</code>	Despliega.
<code>.slideUp()</code>	Pliega.
<code>.slideToggle()</code>	Alterna.
<code>.animate()</code>	Animación personalizada.
<code>.delay()</code>	Retraso.
<code>.stop()</code>	Para animaciones.

Ejemplo:

```
$("#caja").fadeOut(1000);
```

6. Eventos

Eventos comunes del navegador.

Evento	Cuándo ocurre
<code>click</code>	Click.
<code>dblclick</code>	Doble click.
<code>mouseover</code>	Ratón entra encima.
<code>mouseout</code>	Ratón sale.
<code>mousemove</code>	Ratón se mueve.
<code>mousedown</code>	Botón del ratón pulsado.
<code>mouseup</code>	Botón soltado.
<code>keydown</code>	Tecla presionada.
<code>keypress</code>	Tecla imprimible. Antiguo.
<code>keyup</code>	Tecla soltada.
<code>input</code>	Cambia valor mientras se escribe.
<code>change</code>	Cambia valor y se confirma.
<code>submit</code>	Envío de formulario.
<code>focus</code>	Elemento recibe foco.

Evento	Cuándo ocurre
<code>blur</code>	Elemento pierde foco.
<code>load</code>	Recurso cargado.
<code>resize</code>	Cambia tamaño ventana.
<code>scroll</code>	Scroll.
<code>select</code>	Selección de texto.

Ejemplo:

```
form.addEventListener("submit", function(e) {
    e.preventDefault();
    console.log("Formulario validado");
});
```

7. Servlets

Un servlet es una clase Java que responde a peticiones HTTP.

7.1. Clases y objetos principales

Elemento	Uso
<code>HttpServlet</code>	Clase base de los servlets.
<code>HttpServletRequest</code>	Petición recibida.
<code>HttpServletResponse</code>	Respuesta enviada.
<code>HttpSession</code>	Sesión del usuario.
<code>Cookie</code>	Cookie HTTP.
<code>RequestDispatcher</code>	Hace <code>forward</code> o <code>include</code> .
<code>ServletContext</code>	Contexto global de la app.

Ejemplo:

```
public class LoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
```

```
}  
}
```

7.2. Métodos HTTP

Método	Uso
<code>doGet()</code>	Peticiones GET.
<code>doPost()</code>	Peticiones POST.

Ejemplo:

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {  
  
    String usuario = request.getParameter("usuario");  
}
```

7.3. Request

Método	Uso	Ejemplo
<code>getParameter()</code>	Lee dato de formulario/URL.	<code>request.getParameter("nombre")</code>
<code>setAttribute()</code>	Guarda dato en request.	<code>request.setAttribute("msg", "OK")</code>
<code>getAttribute()</code>	Lee atributo.	<code>request.getAttribute("msg")</code>
<code>getCookies()</code>	Lee cookies.	<code>request.getCookies()</code>
<code>getSession()</code>	Obtiene sesión.	<code>request.getSession()</code>

Ejemplo:

```
String nombre = request.getParameter("nombre");  
request.setAttribute("nombre", nombre);
```

7.4. Response

Método	Uso
<code>sendRedirect()</code>	Redirige y cambia URL.

Método	Uso
<code>sendError()</code>	Envía error HTTP.
<code>addCookie()</code>	Añade cookie.
<code>setContentType()</code>	Tipo de respuesta.

Ejemplo:

```
response.sendRedirect(request.getContextPath() + "/home.jsp");
```

7.5. Forward, include y redirect

Método	Cambia URL	Misma request	Uso
<code>forward()</code>	No	Sí	Servlet pasa a JSP.
<code>include()</code>	No	Sí	Incluir otro recurso y volver.
<code>sendRedirect()</code>	Sí	No	Nueva petición del navegador.

Ejemplo:

```
RequestDispatcher rd = request.getRequestDispatcher("/vista.jsp");
rd.forward(request, response);
```

7.6. Sesión y cookies

Crear sesión:

```
HttpSession session = request.getSession();
session.setAttribute("usuario", usuario);
```

Leer sesión:

```
Usuario u = (Usuario) session.getAttribute("usuario");
```

Crear cookie:

```
Cookie c = new Cookie("tema", "oscuro");
c.setMaxAge(60 * 60 * 24);
```

```
response.addCookie(c);
```

Leer cookies:

```
Cookie[] cookies = request.getCookies();
```

7.7. web.xml

Elementos comunes:

Elemento	Uso
<code><web-app></code>	Raíz del descriptor.
<code><servlet></code>	Declara servlet.
<code><servlet-name></code>	Nombre lógico.
<code><servlet-class></code>	Clase Java del servlet.
<code><servlet-mapping></code>	Mapeo URL.
<code><url-pattern></code>	Ruta del servlet.
<code><welcome-file-list></code>	Página inicial.
<code><welcome-file></code>	Archivo de bienvenida.
<code><context-param></code>	Parámetro global.
<code><init-param></code>	Parámetro de servlet.
<code><session-config></code>	Configuración sesión.
<code><session-timeout></code>	Minutos de inactividad.
<code><error-page></code>	Página de error.
<code><error-code></code>	Código HTTP.
<code><location></code>	Página destino.

Ejemplo:

```
<servlet>
  <servlet-name>Login</servlet-name>
  <servlet-class>daw.LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>Login</servlet-name>
```

```
<url-pattern>/login</url-pattern>
</servlet-mapping>
```

8. JSP

JSP permite mezclar HTML con contenido dinámico generado en servidor.

8.1. Directivas JSP

Directiva	Uso	Ejemplo
<code><%@ page %></code>	Configura la página.	<code><%@ page contentType="text/html; charset=UTF-8" %></code>
<code><%@ taglib %></code>	Declara librerías, como JSTL.	<code><%@ taglib prefix="c" uri="..." %></code>
<code><%@ include %></code>	Incluye archivo en tiempo de traducción.	<code><%@ include file="header.jsp" %></code>

Ejemplo:

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ page import="daw.Usuario" %>
```

8.2. Scriptlets y sintaxis clásica

Sintaxis	Nombre	Uso
<code><% ... %></code>	Scriptlet	Código Java.
<code><%= ... %></code>	Expresión	Imprime valor.
<code><%! ... %></code>	Declaración	Declara método o atributo.

Ejemplo scriptlet:

```
<%
String nombre = request.getParameter("nombre");
%>

<p>Nombre: <%= nombre %></p>
```

Declaración:

```
<%!  
public String saludar(String nombre) {  
    return "Hola " + nombre;  
}  
%>  
  
<p><%= saludar("Ana") %></p>
```

Hoy se recomienda evitar scriptlets si se puede, pero en exámenes suelen pedirlos.

8.3. Acciones JSP

Acción	Uso
<code><jsp:include page=""></code>	Incluye otra JSP en ejecución.
<code><jsp:forward page=""></code>	Pasa control a otra JSP/servlet.
<code><jsp:useBean></code>	Crea o recupera bean.
<code><jsp:setProperty></code>	Asigna propiedad.
<code><jsp:getProperty></code>	Imprime propiedad.

Ejemplo bean:

```
<jsp:useBean id="usuario" class="daw.Usuario" scope="request" />  
<jsp:setProperty name="usuario" property="nombre" value="Ana" />  
  
<p>  
    <jsp:getProperty name="usuario" property="nombre" />  
</p>
```

Con formulario:

```
<jsp:useBean id="usuario" class="daw.Usuario" scope="request" />  
<jsp:setProperty name="usuario" property="*" />
```

Esto rellena propiedades si los `name` del formulario coinciden con los setters.

8.4. Objetos implícitos JSP clásicos

Objeto	Uso
<code>request</code>	Petición HTTP.
<code>response</code>	Respuesta HTTP.
<code>session</code>	Sesión del usuario.
<code>application</code>	Contexto global.
<code>out</code>	Escribir salida.
<code>pageContext</code>	Contexto de página.
<code>config</code>	Configuración del servlet/JSP.
<code>page</code>	La página actual.
<code>exception</code>	Excepción, solo en páginas de error.

Ejemplo:

```
<%  
String nombre = request.getParameter("nombre");  
session.setAttribute("usuario", nombre);  
%>
```

9. EL

EL significa **Expression Language**. Sirve para acceder a datos de forma limpia con `${...}`.

9.1. Acceso básico

Servlet:

```
request.setAttribute("usuario", usuario);
```

JSP:

```
<p>${usuario.nombre}</p>
```

Eso llama internamente a:

```
usuario.getNombre()
```

9.2. Objetos implícitos comunes de EL

Objeto	Uso	Ejemplo
<code>pageScope</code>	Atributos de página.	<code>\${pageScope.dato}</code>
<code>requestScope</code>	Atributos de request.	<code>\${requestScope.usuario.nombre}</code>
<code>sessionScope</code>	Atributos de sesión.	<code>\${sessionScope.usuario.nombre}</code>
<code>applicationScope</code>	Atributos globales.	<code>\${applicationScope.contador}</code>
<code>param</code>	Parámetros simples.	<code>\${param.nombre}</code>
<code>paramValues</code>	Parámetros múltiples.	<code>\${paramValues.aficion[0]}</code>
<code>header</code>	Cabeceras HTTP.	<code>\${header["User-Agent"]}</code>
<code>headerValues</code>	Cabeceras múltiples.	<code>\${headerValues.accept[0]}</code>
<code>cookie</code>	Cookies.	<code>\${cookie.userID.value}</code>
<code>initParam</code>	Parámetros de contexto.	<code>\${initParam.moda}</code>
<code>pageContext</code>	Contexto JSP.	<code>\${pageContext.request.contextPath}</code>

Ejemplos:

```
${param.nombre}
```

lee un parámetro de formulario o URL.

```
${requestScope.usuario.nombre}
```

lee un atributo de request.

```
${sessionScope.usuario.nombre}
```

lee un atributo de sesión.

```
${cookie.tema.value}
```

lee una cookie.

```
${pageContext.request.contextPath}
```

obtiene la ruta base de la aplicación.

9.3. Scopes

Scope	Duración	Ejemplo
pageScope	Solo JSP actual.	Variable local.
requestScope	Una petición.	Datos de servlet a JSP.
sessionScope	Varias peticiones del usuario.	Usuario logueado.
applicationScope	Toda la aplicación.	Contador global.

Si escribes:

```
${usuario}
```

EL busca en este orden:

```
pageScope → requestScope → sessionScope → applicationScope
```

9.4. HashMap en EL

Si tienes:

```
HashMap<String, String> h = new HashMap<>();  
h.put("nombre", "Ana");  
request.setAttribute("datos", h);
```

Puedes hacer:

```
${datos.nombre}
```

o:

```
${datos["nombre"]}
```

Si la clave tiene espacios o caracteres raros, mejor corchetes:

```
${datos["nombre completo"]}
```

10. JSTL

JSTL permite usar etiquetas en JSP para condiciones, bucles y salida.

Primero se declara:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

o en Jakarta:

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

10.1. Salida y variables

Etiqueta	Uso	Ejemplo
<code><c:out></code>	Imprime valor escapado.	<code><c:out value="\${usuario.nombre}" /></code>
<code><c:set></code>	Crea/asigna variable.	<code><c:set var="x" value="10" /></code>

Ejemplo:

```
<c:set var="saludo" value="Hola" />
<p><c:out value="${saludo}" /></p>
```

10.2. Condicional simple

```
<c:if test="${edad >= 18}">
  <p>Mayor de edad</p>
</c:if>
```

10.3. Condicional múltiple

```
<c:choose>
  <c:when test="${nota >= 9}">
    <p>Sobresaliente</p>
  </c:when>

  <c:when test="${nota >= 5}">
    <p>Aprobado</p>
  </c:when>

  <c:otherwise>
    <p>Suspenso</p>
  </c:otherwise>
</c:choose>
```

Equivale a:

```
if (...) {
} else if (...) {
} else {
}
```

10.4. Bucles

Bucle numérico:

```
<c:forEach var="i" begin="1" end="5">
  <p>${i}</p>
</c:forEach>
```

Bucle sobre lista:

```
<c:forEach var="usuario" items="${usuarios}">
  <p>${usuario.nombre}</p>
</c:forEach>
```

Con HashMap:

```
<c:forEach var="entrada" items="${datos}">
  <p>${entrada.key}: ${entrada.value}</p>
</c:forEach>
```

11. Java básico para Servlets/JSP

11.1. List

```
List<String> lista = new ArrayList<>();
lista.add("Ana");
lista.add("Luis");

String primero = lista.get(0);
```

Uso típico: lista ordenada de elementos.

11.2. ArrayList

```
ArrayList<Usuario> usuarios = new ArrayList<>();
usuarios.add(usuario);
```

Permite repetidos.

11.3. Vector

```
Vector<String> v = new Vector<>();
v.add("A");
v.get(0);
```

Parecido a `ArrayList`, pero sincronizado. Es más antiguo.

11.4. HashMap

```
HashMap<String, Usuario> h = new HashMap<>();  
  
h.put("u1", usuario);  
Usuario u = h.get("u1");
```

Usa pares clave-valor.

Importante:

```
h.put("u1", usuario1);  
h.put("u1", usuario2);
```

no crea duplicado. Sustituye el valor anterior.

12. Mini chuleta final

HTML

```
<form action="Servlet" method="post">  
  <input type="text" name="nombre">  
  <button type="submit">Enviar</button>  
</form>
```

CSS

```
.caja {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

JavaScript

```
const p = document.querySelector("p");  
p.textContent = "Hola";
```

Evento

```
boton.addEventListener("click", function(e) {  
    e.preventDefault();  
});
```

jQuery

```
$("#p").on("click", function() {  
    $(this).css("color", "red");  
});
```

Servlet

```
String nombre = request.getParameter("nombre");  
request.setAttribute("nombre", nombre);  
request.getRequestDispatcher("/vista.jsp").forward(request, response);
```

JSP scriptlet

```
<%  
String nombre = (String) request.getAttribute("nombre");  
%>  
<p><%= nombre %></p>
```

JSP con EL

```
<p>${requestScope.nombre}</p>
```

JSTL

```
<c:if test="${not empty nombre}">  
    <p>${nombre}</p>  
</c:if>
```

JavaBean

```
<jsp:useBean id="usuario" class="daw.Usuario" scope="request" />  
<jsp:getProperty name="usuario" property="nombre" />
```

Preguntas

Recopilación basada en los PDFs proporcionados por el usuario.

Cuando el documento trae respuesta, se incluye.

Cuando no aparece una respuesta explícita, se indica: **Respuesta en documento: no aparece.**

No se han inventado respuestas fuera del contenido de los documentos.

Parcial DAW 2026

1. ¿Qué significa SPA y MPA? Completa la tabla.

Respuesta en documento: sí aparece tabla.

Característica	MPA	SPA
Navegación	Página completa se recarga	Sin recarga, solo cambia el contenido
SEO	Muy bueno	Requiere SSR o trucos
Velocidad inicial	Alta	A veces lenta, mucho JS
Experiencia de usuario	Más tradicional	Más fluida, tipo app
Complejidad	Baja	Mayor: estado, ruteo, API...
Carga del servidor	Alta	Más baja
Comunicación	Peticiones HTTP tradicionales	AJAX, APIs: REST, GraphQL
Ventajas	Mejor SEO, fácil escalabilidad en contenido	Tipo app, menos ancho de banda tras carga inicial
Desventajas	Experiencia menos fluida, más ancho de banda	SEO difícil, carga inicial lenta, más JS
Casos de uso	Blogs, e-commerce, periódicos, sitios corporativos	Dashboards, Gmail, Trello, apps móviles/web

2. ¿Cuáles son las 3 características de HTML?

Las tres propiedades principales indicadas son:

1. Universalidad:

- 2. **Lenguaje de marcas:** permite organizar el contenido usando etiquetas adecuadas para indicar su función.
- 3. **Hipertexto:** permite crear enlaces entre documentos o secciones.

3. ¿Cómo se indica la versión HTML de un documento? Pon como se indica que un documento es HTML5.

La versión de HTML de un documento se indica mediante la declaración **DOCTYPE**, que se coloca al principio del archivo, antes de la etiqueta `<html>`.

En HTML5 se simplificó y se indica así:

```
<!DOCTYPE html>
```

4. Explica los posicionamientos absoluto, fijo y relativo.

Puesto un 0,6 en el parcial

Posicionamiento **absoluto**: es aquel que se sale del flujo y se posiciona respecto al elemento padre (siempre que esté no tenga `position:static`, en cuyo caso se posicionará respecto al ancestro que no cumpla esta condición).

```
[A] [B] [C]
      |
      v
[A] [C]
      [B]
```

Posicionamiento **fijo**: es aquel que se sale del flujo de forma que se posiciona respecto a la pantalla/viewport y es visible en todo momento aunque haya scroll.

Posicionamiento **relativo**: es aquel que **NO** se sale del flujo, se posiciona a partir del sitio que debería ocupar en el flujo

```
[A] [B] [C]
      |
      v
```

5. Explica cuál es el cometido de CSS y por qué se introdujo por el W3C. ¿Cómo es la sintaxis de CSS? Nombra sus componentes y pon un ejemplo.

CSS significa **Cascading Style Sheets** y se introdujo en HTML 4.0 y fue propuesto por W3C para separar contenidos y formato de una web. Su cometido principal es definir el aspecto de los elementos HTML: colores, tamaños, márgenes, bordes, fuentes, distribución en pantalla, posicionamiento, etc.

CSS tiene un **selector** y un conjunto de reglas/declaraciones. El selector → determina qué elementos serán afectados. La declaración → uno o más pares propiedad/valor separados por ;.

```
div p {  
  color: lime;  
}
```

7. Explica los dos grandes grupos de etiquetas que existen en HTML.

Existen **2 tipos de etiquetas**:

- **Etiquetas de bloque**
- **Etiquetas de línea**

◆ Etiquetas de bloque

- Ocupan toda la línea
- Provocan salto de línea antes y después
- Ej: `h1`, `p`, listas

◆ Etiquetas de línea

- No ocupan toda la línea
- No provocan salto de línea
- Se adaptan al contenido
- Ej: `a`, `img`, `strong`

9. ¿Qué aporta HTML 4.0?

HTML 4.0 aporta la separación entre **contenido/estructura** y **formato**, mediante CSS.

11. ¿Qué significa la etiqueta `<meta name="robots" content="noindex, nofollow">`?

Directiva para los robots de los motores de búsqueda e indica que la página **no debe ser indexada ni mostrada en los resultados de búsqueda** (`noindex`) y que, además, **no deben seguirse los enlaces** que contiene esa página (`nofollow`). En conjunto, sirve para evitar que esa página aparezca en buscadores y para impedir que los rastreadores continúen navegando a través de sus enlaces.

13. ¿Para qué se utiliza `@import`?

`@import` sirve para importar otro archivo CSS externo y aplicar sus estilos al documento actual.

Ejemplos dados:

```
@import url("estilos.css");
```

O:

```
@import "estilos.css";
```

14. ¿Qué hace la propiedad `box-sizing` y qué valores puede tomar?

La propiedad `box-sizing` indica **cómo se calcula el tamaño total de una caja** en CSS.

- `content-box`: `width` y `height` solo incluyen el contenido. El `padding` y el `border` se suman aparte.
 - `border-box`: `width` y `height` incluyen contenido, `padding` y `border`.
-

15. ¿De qué librería es esta regla y cómo afecta a la renderización?

```
<p class="col-sm-8">
```

La librería/framework es **Bootstrap**. Usa un sistema responsivo de **12 columnas**.

La clase `col-sm-8` indica que, en pantallas de tamaño **small (sm)** o **superior**, ese elemento ocupará **8 columnas de las 12 disponibles**. Es decir, ocupará aproximadamente **dos tercios del ancho** de la fila.

En pantallas más pequeñas que `sm`, si no se especifica otra clase, el elemento suele colocarse ocupando todo el ancho disponible, apilándose con otros elementos.

16. Diferencia entre `HTMLCollection` y `NodeList`.

- Se puede acceder a los elementos de **HTMLCollection** por su nombre, **id** o índice, pero a los de **NodeList** sólo se puede acceder por su índice.
 - Sólo el objeto **NodeList** puede contener nodos de atributo y nodos de texto.
 - **HTMLCollection** es viva y **NodeList** es estática, es decir, el primero reconocerá automáticamente un nuevo elemento añadido al DOM mientras que el otro no
-

17. Define aplicación web.

Una aplicación web es un conjunto de páginas web generadas en respuesta a peticiones del usuario, típicamente mediante formularios, y necesita tecnologías de front-end y back-end.

También aparece una respuesta de examen: software que se ejecuta en un navegador web y ofrece funcionalidades similares a una aplicación de escritorio, accediéndose por internet sin instalación local.

18. Definición de servlet.

Un servlet es un programa escrito en Java que se ejecuta en el contenedor web de un servidor de aplicaciones y es invocado por un cliente mediante HTTP.

19. Definición de JSP: siglas y qué son.

JSP significa **Java Server Pages**. Son páginas HTML con líneas de código adicional que se ejecutan en el servidor para generar contenido dinámico. Se traducen automáticamente a servlets.

20. Definición de JavaBean.

Un JavaBean es una clase Java que mantiene datos llamados propiedades, cuyos valores se recuperan/asignan mediante getters y setters.

Debe tener:

- constructor sin argumentos,
 - getters/setters para atributos privados,
 - implementación de `Serializable`.
-

21. ¿Qué significa que una etiqueta sea de línea en HTML?

Una etiqueta de línea, o inline, no inicia una nueva línea de texto. Se coloca dentro de una línea existente y no crea salto de línea adicional.

Ejemplos indicados:

- `<span>`
 - `<strong>`
 - `<a>`
-

22. ¿Qué es W3C? ¿Quién lo creó y para qué?

El **W3C**, World Wide Web Consortium, es una organización internacional que establece estándares y pautas para el desarrollo web.

Fue creado por **Tim Berners-Lee** en 1994 para promover la interoperabilidad y el crecimiento ordenado de la World Wide Web.

25. Diferencias y similitudes entre Mocha, JScript y VBScript.

Respuesta en documento: sí aparece.

Mocha aparece como el nombre original de JavaScript. JScript y VBScript se presentan como versiones desarrolladas por Microsoft para Internet Explorer.

VBScript es incompatible con navegadores diferentes a Internet Explorer y tiene sintaxis basada en Visual Basic.

26. Explica el funcionamiento de `addEventListener`. ¿Cuál es el significado de los parámetros de `addEventListener`?

`addEventListener` sirve para asociar un evento a un elemento HTML. Permite detectar y responder a acciones del usuario como:

- `click`
- `mouseover`
- `keydown`
- `submit`

Sintaxis:

```
element.addEventListener(event, callback, useCapture);
```

Parámetros:

- `event`: cadena con el tipo de evento.
- `callback`: función que se ejecuta cuando ocurre el evento.
- `useCapture`: booleano opcional que indica si se usa captura en lugar de bubbling.

28. ¿Qué es Java EE? Escribe tres tecnologías que incluye Java EE.

Java EE aparece como la especificación usada para el desarrollo web, construida sobre Java SE/JDK, que describe cómo los servidores pueden interactuar con tecnologías web Java.

En los apuntes de servlets aparecen como APIs de Java EE:

- Servlets.
- JSP.
- JSF.
- JPA.
- EJB.

30. ¿Qué papel juegan `HttpServletRequest` y `HttpServletResponse` en un servlet?

`HttpServletRequest` representa la solicitud HTTP recibida por el servidor e incluye parámetros, cabeceras y cuerpo de la solicitud.

`HttpServletResponse` representa la respuesta enviada al cliente y permite configurar cabeceras, escribir el cuerpo de la respuesta o redirigir.

31. ¿Qué es un JSP y para qué sirve?

Un JSP es una página HTML con código adicional Java que se ejecuta en el servidor para generar contenido dinámico. Sirve para crear páginas dinámicas y separar la presentación de parte de la lógica Java.

32. ¿Para qué sirve la directiva `@page`?

`@page` define características aplicables a toda la página JSP, como:

- lenguaje,
- imports,

- sesión,
 - buffer,
 - `contentType`,
 - `pageEncoding`,
 - si se ignora EL.
-

33. ¿Para qué sirve la directiva `@tagLib`?

`@tagLib` sirve para utilizar elementos definidos en librerías estándar o propias, asociando una URI de librería a un prefijo.

35. ¿Qué implicaciones tuvo la aparición del iPhone 3 y el Mobile-First Indexing de Google en el front-end?

Respuesta en documento: sí aparece.

El iPhone impulsó la navegación web desde dispositivos móviles, haciendo necesario crear aplicaciones web compatibles y optimizadas para móvil.

Mobile-First Indexing hizo que Google indexase y clasificase sitios según su versión móvil, impulsando enfoques de diseño mobile-first.

37. Diferencias en renderizado entre `display: block`, `display: inline` y `display: inline-block`.

`display: block` hace que el elemento ocupe toda la línea disponible y empiece en una línea nueva; permite modificar `width`, `height`, márgenes y padding.

`display: inline` hace que el elemento se coloque en la misma línea que el texto u otros elementos, ocupando solo lo necesario; no permite controlar bien `width` y `height`.

`display: inline-block` combina ambos: se coloca en línea como un `inline`, pero permite definir `width`, `height`, márgenes y padding como un bloque.

38. Diferencias entre `<head>` y `<header>`, `<foot>` y `<footer>`.

`<head>` contiene información interna del documento que no se muestra directamente en la página, como el título, metadatos, enlaces a CSS o scripts. `<header>` es una etiqueta visible del cuerpo de la página y representa una cabecera de contenido, como el encabezado de una web, una sección o un artículo.

La etiqueta `<foot>` no es una etiqueta HTML estándar; la correcta es `<footer>`, que representa el pie de página o pie de una sección, normalmente con información como autoría, contacto, copyright o enlaces relacionados.

39. ¿Qué es un lenguaje de marcas? Nombra dos ejemplos.

Un lenguaje de marcas es un lenguaje que utiliza etiquetas para estructurar, describir o dar significado al contenido de un documento. No es un lenguaje de programación, porque no ejecuta instrucciones lógicas, sino que marca partes del texto o de los datos. Dos ejemplos son **HTML**, usado para estructurar páginas web, y **XML**, usado para representar y transportar datos estructurados.

40. Relaciona conceptos: HTML, CSS, WHOIS, Hipertexto, Mosaic.

- HTML → Tim Berners-Lee.
 - CSS → HTML 4.0.
 - WHOIS → nombres de dominio.
 - Hipertexto → Ted Nelson.
 - Mosaic → Marc Andreessen.
-

41. ¿Qué framework se utiliza en `<div class="col-sm-4">`? ¿Qué tamaño máximo puede alcanzar esa cifra en Bootstrap? ¿Qué propiedad de CSS usa Bootstrap para ese layout? ¿Para qué sirve el sistema de columnas de Bootstrap?

```
<div class="col-sm-4">
```

pertenece al framework **Bootstrap**.

Bootstrap utiliza un sistema de rejilla responsive basado en **12 columnas**, por lo que la cifra máxima habitual es **12**. En este caso, `col-sm-4` indica que el elemento ocupará **4 columnas de 12** en pantallas de tamaño `sm` o superior.

Bootstrap usa principalmente **Flexbox**, es decir, la propiedad CSS:

```
display: flex;
```

Este sistema sirve para generar páginas **responsivas**, adaptando la distribución de los elementos según el tamaño de pantalla. En pantallas `sm` o superiores se crean columnas; en pantallas más pequeñas, los elementos suelen colocarse unos debajo de otros.

45. ¿Para qué sirve la etiqueta `<div>`?

`<div>` es una etiqueta de bloque que sirve para agrupar conjuntos de etiquetas.

46. ¿Qué dos atributos son los más usados en `<div>` y en qué se diferencian?

Los dos atributos más usados son `id` y `class`.

- `id`: identifica una etiqueta concreta y no debe repetirse en el mismo documento.
- `class`: identifica conjuntos de etiquetas y puede aplicarse a varias etiquetas.

47. ¿Qué son `request` y `response` y para qué sirven?

`request` y `response` son instancias de `HttpServletRequest` y `HttpServletResponse`. Encapsulan la información asociada a la petición y a la respuesta de una conexión HTTP.

Permiten obtener parámetros enviados por formulario y establecer una respuesta.

49. ¿Qué atributo hay que añadir en un formulario para verificar que el email es válido?

Puede usarse:

```
<input type="email">
```

O un `pattern` como:

```
pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2, 4}$"
```

50. ¿Cómo hacer que todos los campos del formulario sean obligatorios?

Con el atributo:

```
<input type="text" name="nombre" required>
```

51. ¿Qué método de intercambio utilizan los formularios y qué implicaciones tiene?

Los formularios usan principalmente `GET` y `POST`: `GET` envía los datos en la URL y se usa para consultas; `POST` los envía en el cuerpo de la petición y se usa para enviar/modificar datos de forma más segura y con mayor tamaño.

```
<form action="procesar" method="get">
```

```
<form action="procesar" method="post">
```

52. ¿Qué debemos teclear para construir el fichero `.war`?

```
jar -cf etseDAW.war /
```

El documento también indica que `c` crea un nuevo archivo JAR y `f` fija el nombre del fichero que se crea.

53. ¿Qué debemos teclear en el navegador para invocar el servlet del ejemplo?

```
https://193.144.75.11:8080/etseDAW/nombreDelServlet
```

54. ¿Qué restricción tiene el campo del formulario ID y por qué?

56. Indica los dos lenguajes de marcas más usados en desarrollo web.

HTML y XML

61. Definición de servlet, JavaBean y JSP.

JavaBean: clase Java que mantiene datos o propiedades, sigue convenciones de codificación y recupera/asigna valores mediante getters y setters.

JSP: páginas HTML con líneas de código adicional ejecutadas en servidor para generar contenido dinámico.

Servlet: en el fragmento de Junio 2024 no aparece la definición escrita, pero sí aparece en DAW_6: programa Java que se ejecuta en un contenedor web y es invocado por HTTP.

62. ¿Quién creó HTML y en qué contexto?

- HTML → Tim Berners-Lee.

Pero no el contexto completo.

65. ¿Cómo indicar la codificación que se quiere usar en una página?

Respuesta en documento: sí aparece para HTML5.

```
<meta charset="UTF-8">
```

67. ¿Cómo se usa `@import`?

```
@import url("estilos.css");
```

O:

```
@import "estilos.css";
```

69. ¿Qué son `id`, `class` y `name` de una etiqueta?

Respuesta en documento: aparece parcialmente.

Para `id` y `class` sí aparece:

- `id`: identifica una etiqueta concreta, sin repetirse.
- `class`: identifica conjuntos de etiquetas, puede repetirse en varios elementos.

Sobre `name`, no aparece respuesta desarrollada en los fragmentos localizados.

70. Definición de JSP, EL y JSTL.

JSP: páginas HTML con código adicional ejecutado en servidor para contenido dinámico.

EL: Expression Language proporciona una sintaxis compacta para manejar JavaBeans, arrays, listas y mapas almacenados como atributos.

JSTL: JavaServer Standard Tag Library amplía la funcionalidad de JavaBeans con etiquetas estándar que permiten eliminar scriptlets de JSP.

71. ¿Cómo indicar que se quieren usar EL y JSTL?

Para EL:

```
<%@ page isELIgnored="false" %>
```

Para JSTL:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

Lo más repetido en exámenes

Estas son las preguntas teóricas más repetidas o más probables según los documentos:

1. **HTML**: siglas, características, HTML5, `DOCTYPE`, etiquetas de línea/bloque.
2. **CSS**: siglas, función, regla CSS, `@import`, `box-sizing`, Bootstrap.
3. **Aplicación web**: definición.
4. **Servlet/JSP/JavaBean**: definiciones.
5. **Java EE**: qué es y tecnologías.
6. **Request/response**: papel en servlets.
7. **JSP/EL/JSTL**: definición y directivas.
8. **addEventListener**: funcionamiento y parámetros.
9. **web.xml / estructura de despliegue**.
10. **SPA vs MPA**.